Universidad Icesi

React Hooks y React Router

Computación en Internet 2 · Semana 15

Universidad Icesi

Universidad ICESI

1. useState

La memoria del componente

Universidad icesi

Un componente necesita dos cosas más

Recordar
Guardar un valor entre un render y otro: **useState**.

Conectarse
Sincronizarse con lo que React no controla (el servidor, el título de la página): **useEffect**.

Reutilizar
Sacar ambos a tus propias funciones: **custom hooks**.

Los **hooks** son funciones que le dan esas capacidades a un componente. Su nombre siempre empieza con `use`.

Un contador con una variable normal

Contador.jsx

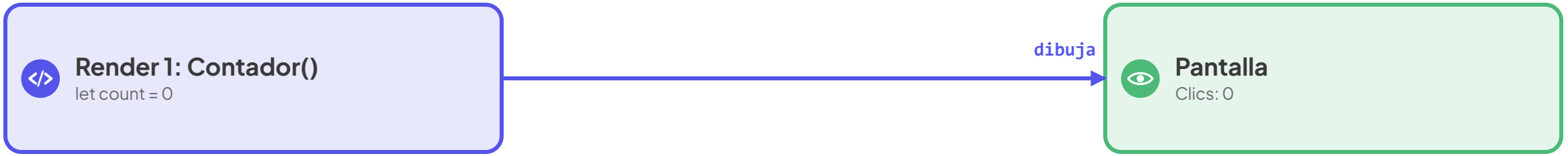
```jsx
function Contador() {
  let count = 0;

  return (
    <button onClick={() => { count = count + 1; }}>
      Clics: {count}
    </button>
  );
}
```

Nunca cambia en pantalla
El botón se queda en «Clics: 0».

Dos razones
Las vemos en los siguientes pasos.

Las variables locales no sobreviven al render



React ejecuta la función `Contador` y dibuja lo que devuelve. `count` es una variable local: nace con el valor 0.

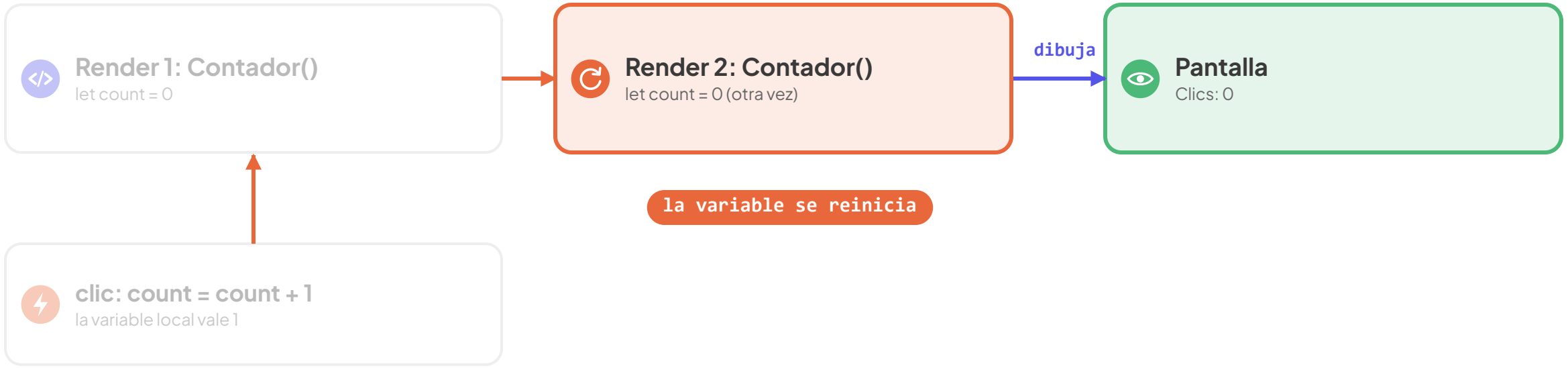
Cambiarla no le avisa a React



El clic sube `count` a 1, pero **nada le dice a React que dibuje otra vez**: la pantalla sigue en 0.

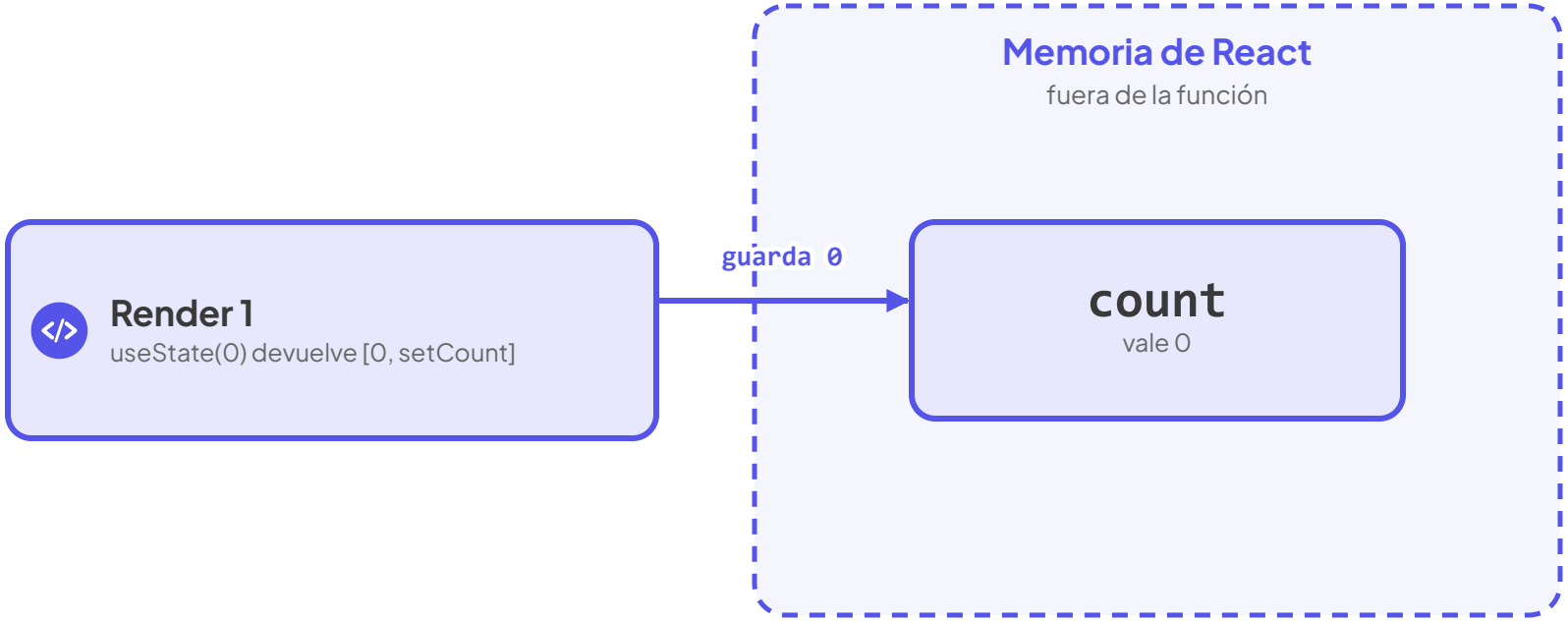
Universidad ICESI

Y si React vuelve a ejecutarla, empieza de cero



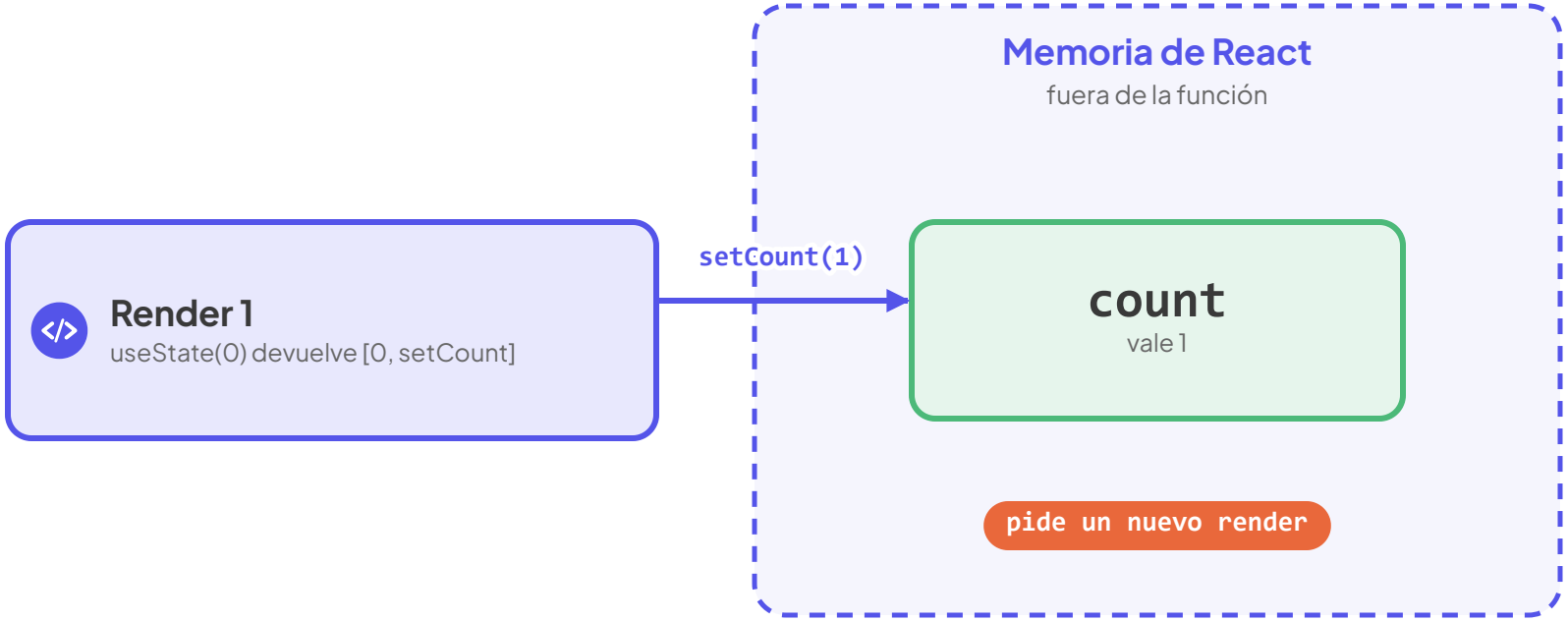
Cada render es una llamada nueva a la función: `count` vuelve a ser 0. Hace falta guardar el valor **fuera** de la función.

useState guarda el valor fuera de la función



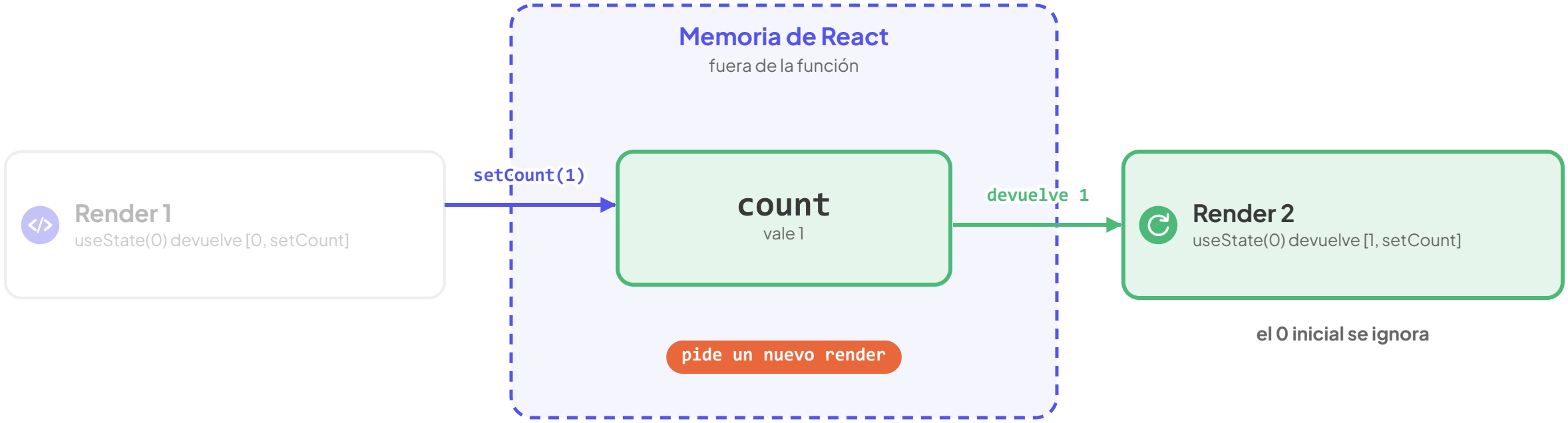
En el primer render, `useState(0)` guarda el 0 en la **memoria de React** y devuelve el valor y una función para cambiarlo.

setCount cambia el valor y pide otro render



El clic llama a `setCount(1)` : React actualiza lo que tiene guardado y **vuelve a ejecutar** el componente.

En el siguiente render, useState devuelve el valor guardado



El argumento `0` solo se usa la primera vez. Ahora `useState` devuelve 1: el valor **sobrevivió** entre renders.

Universidad ICESI

useState devuelve dos cosas

Contador.jsx

```jsx
import { useState } from "react";

function Contador() {
  const [count, setCount] = useState(0);

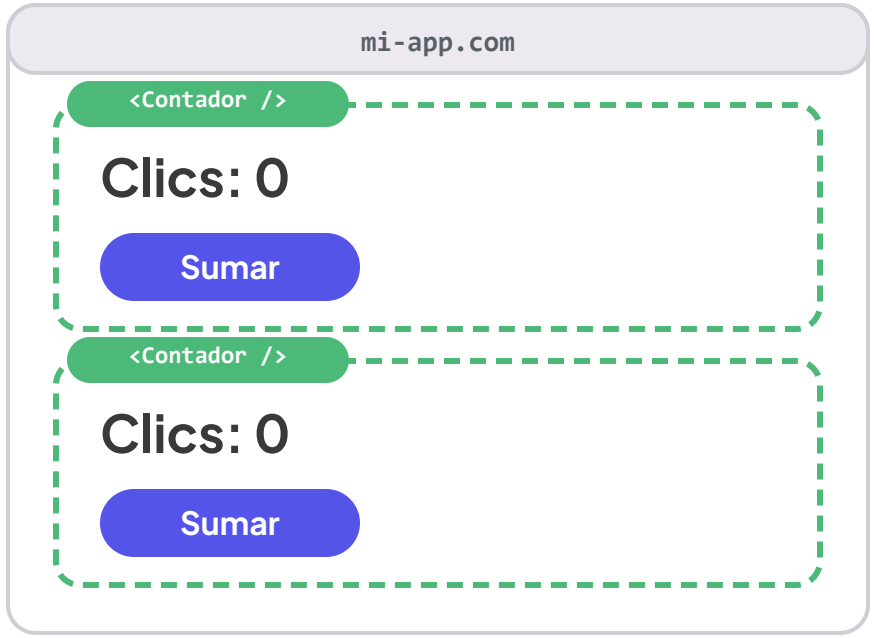
  return (
    <button onClick={() => setCount(count + 1)}>
      Clics: {count}
    </button>
  );
}
```

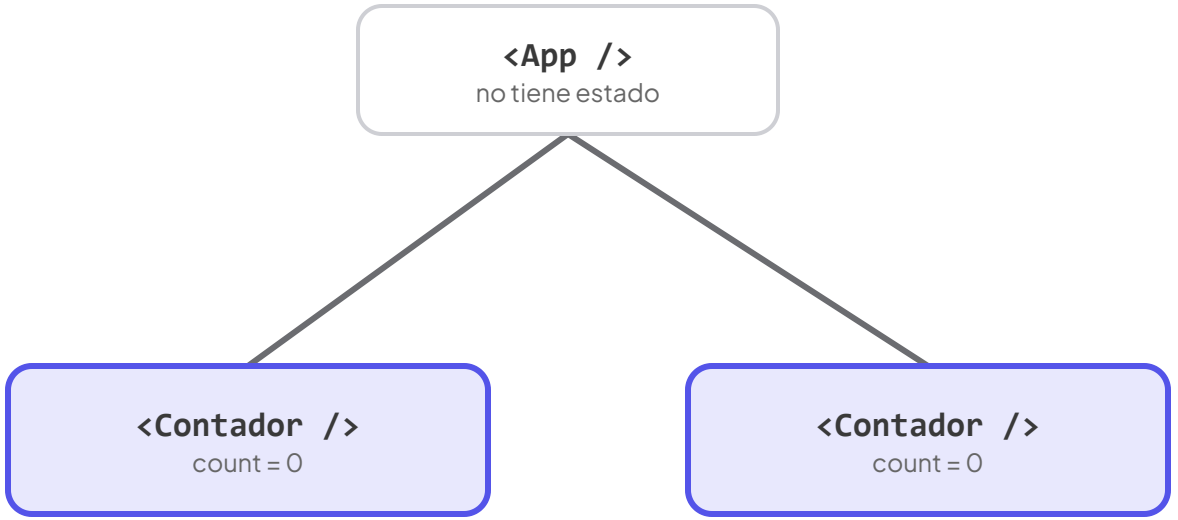
count
La **variable de estado**: el valor actual.

setCount
El **setter**: guarda un valor nuevo y pide otro render.

useState(0)
El 0 es el valor inicial, solo para el primer render.

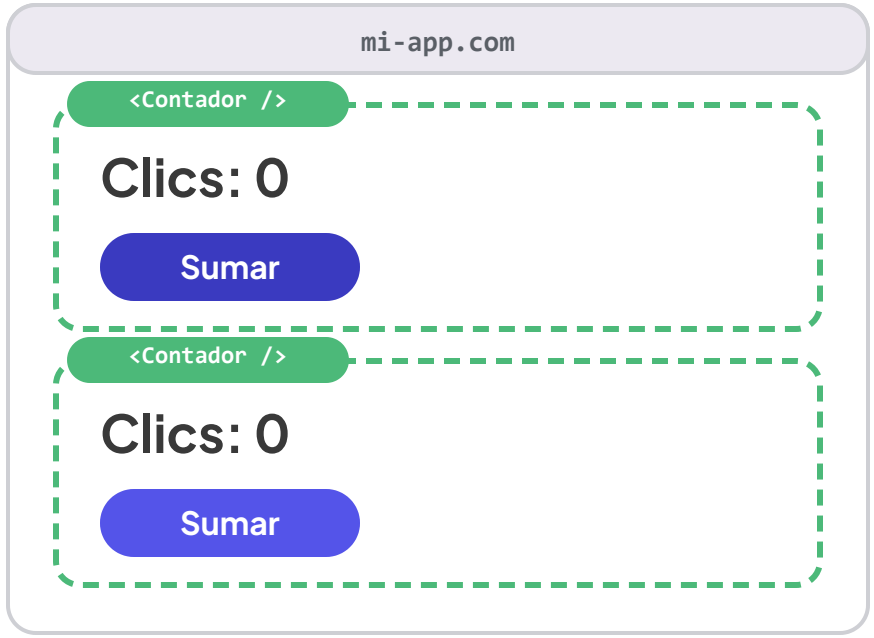
React ejecuta Contador una vez por copia

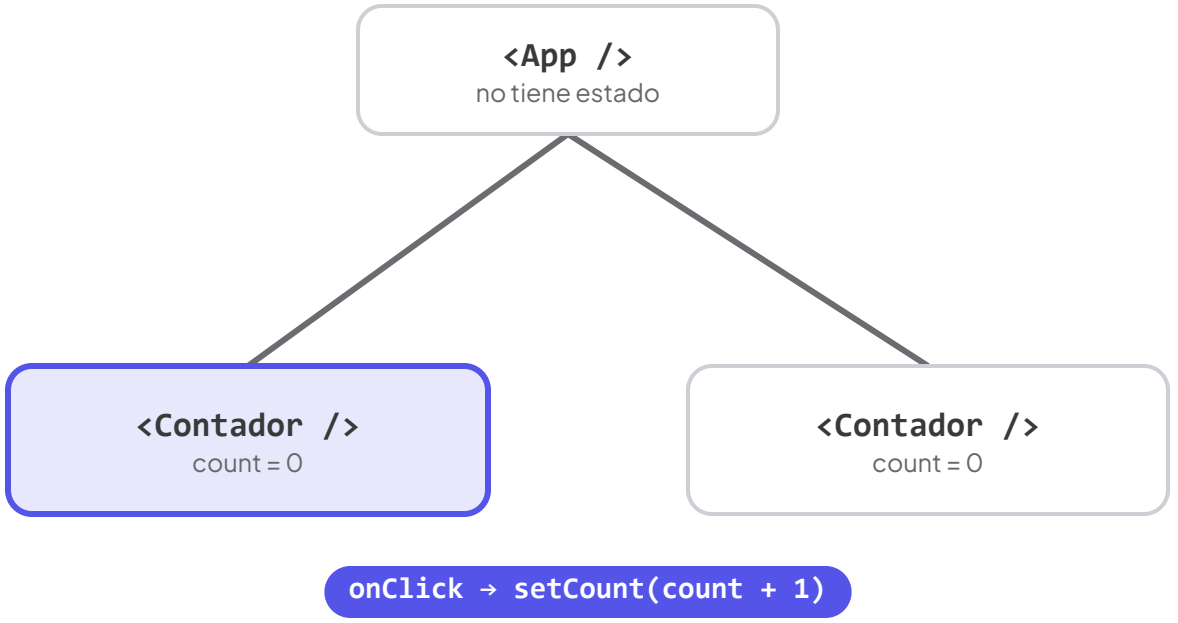




Dos copias del mismo componente dentro de `App`. En el primer render React llama a la función dos veces: cada llamada recibe **su propio** `count`.

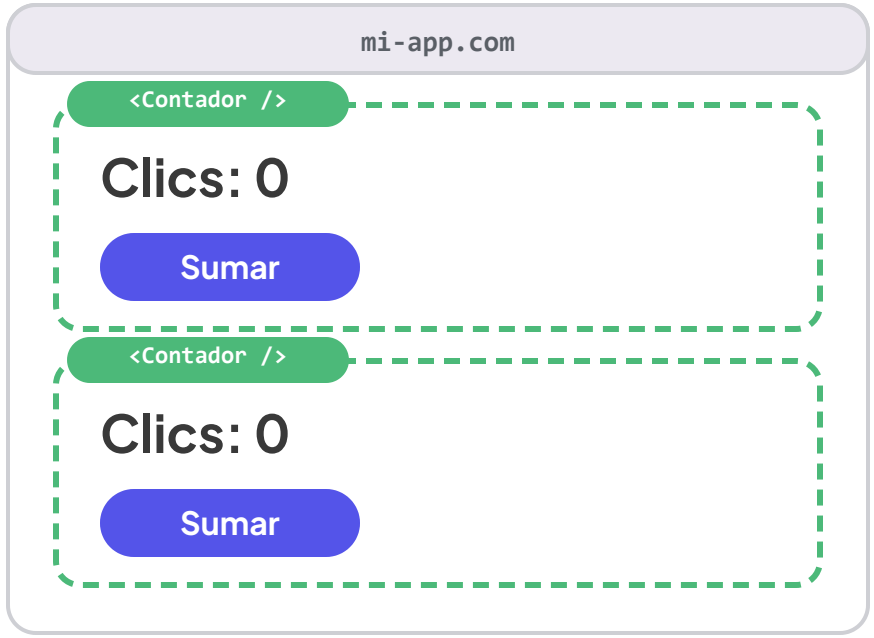
El usuario pulsa el primer botón

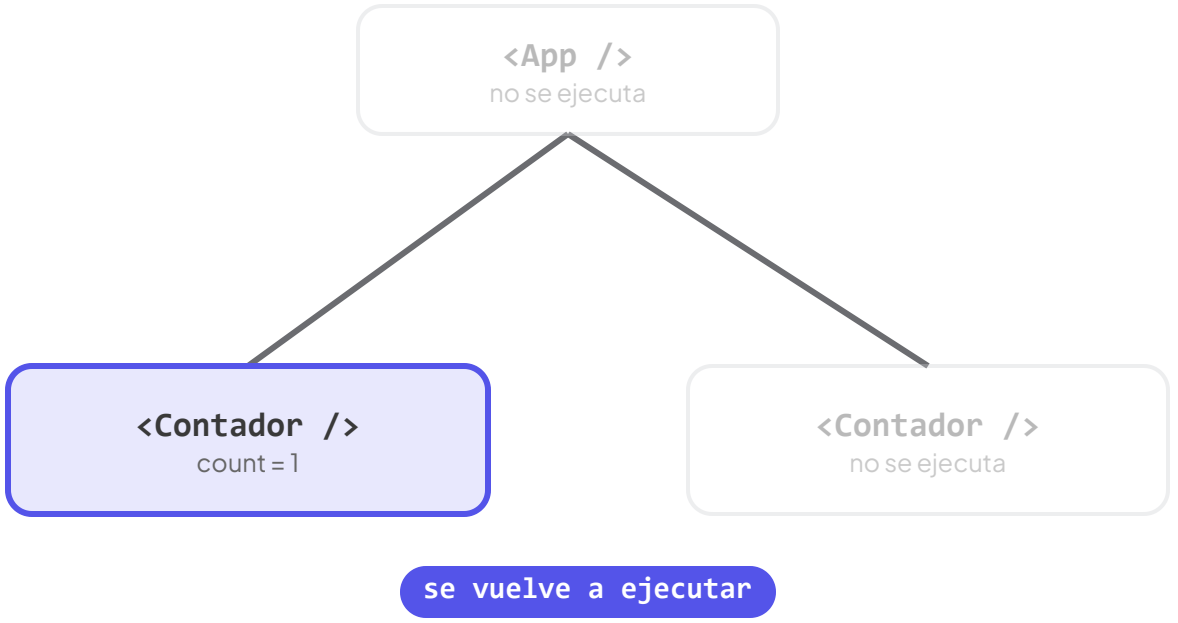




El manejador llama a `setCount`: pide a React un valor nuevo **para esa copia**. La pantalla todavía no cambió.

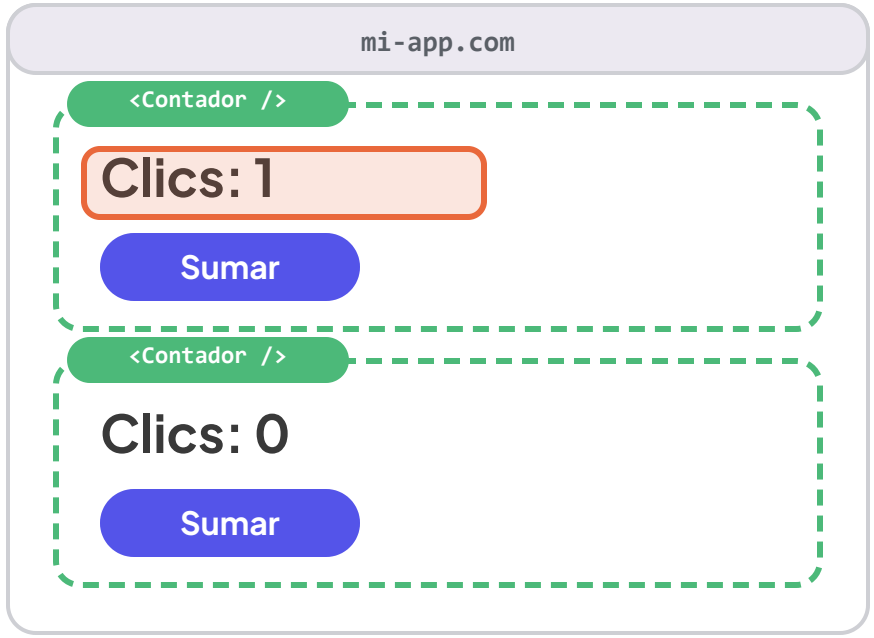
React vuelve a ejecutar solo esa copia

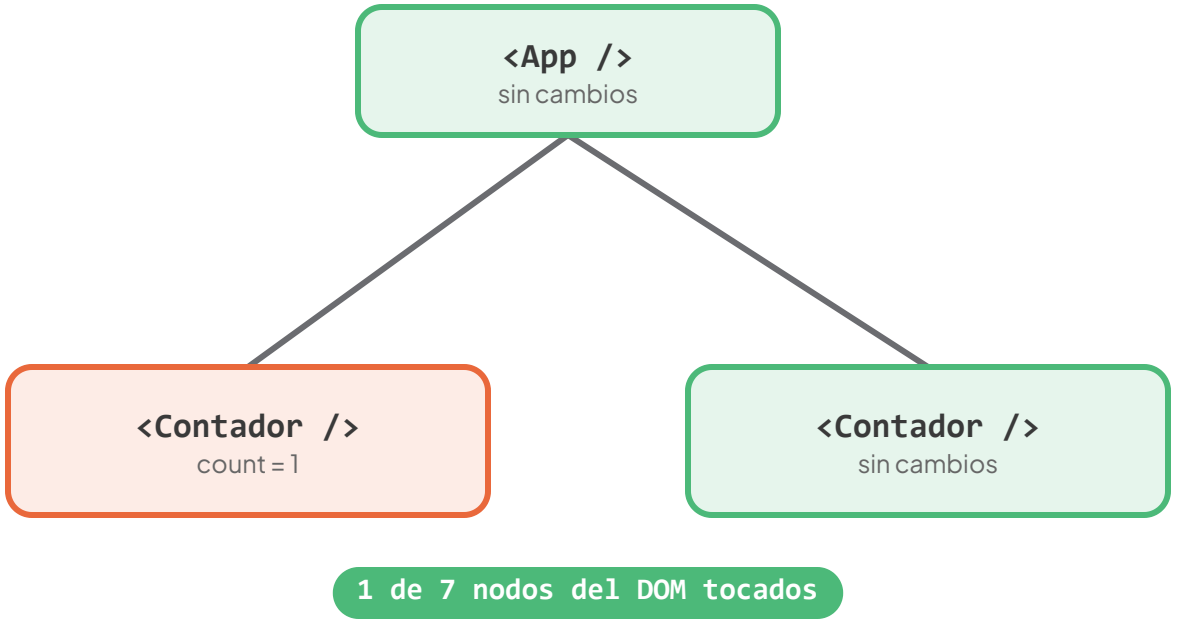




Solo el componente que cambió se ejecuta de nuevo, ahora con `count = 1`. El padre y la otra copia no se tocan.

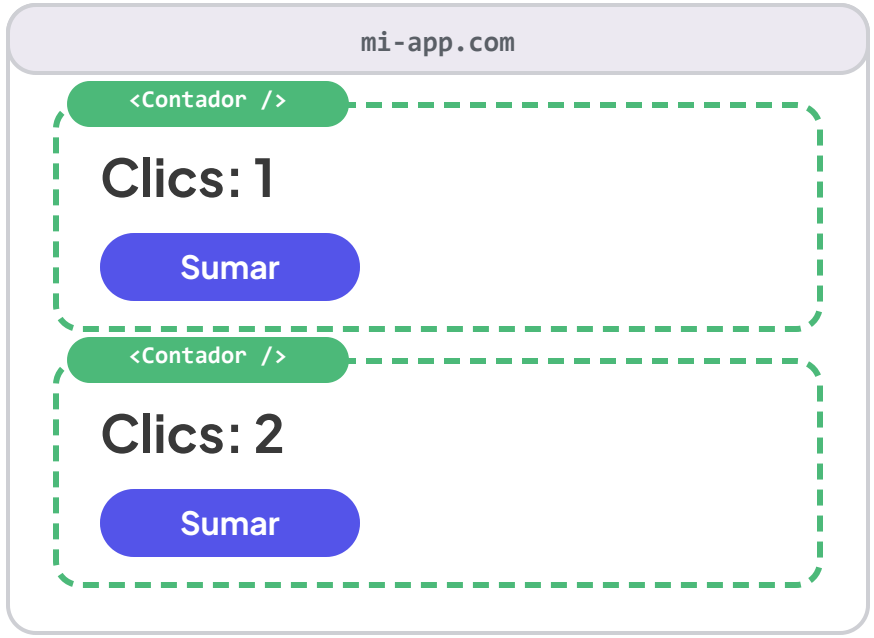
Solo cambia un nodo del DOM

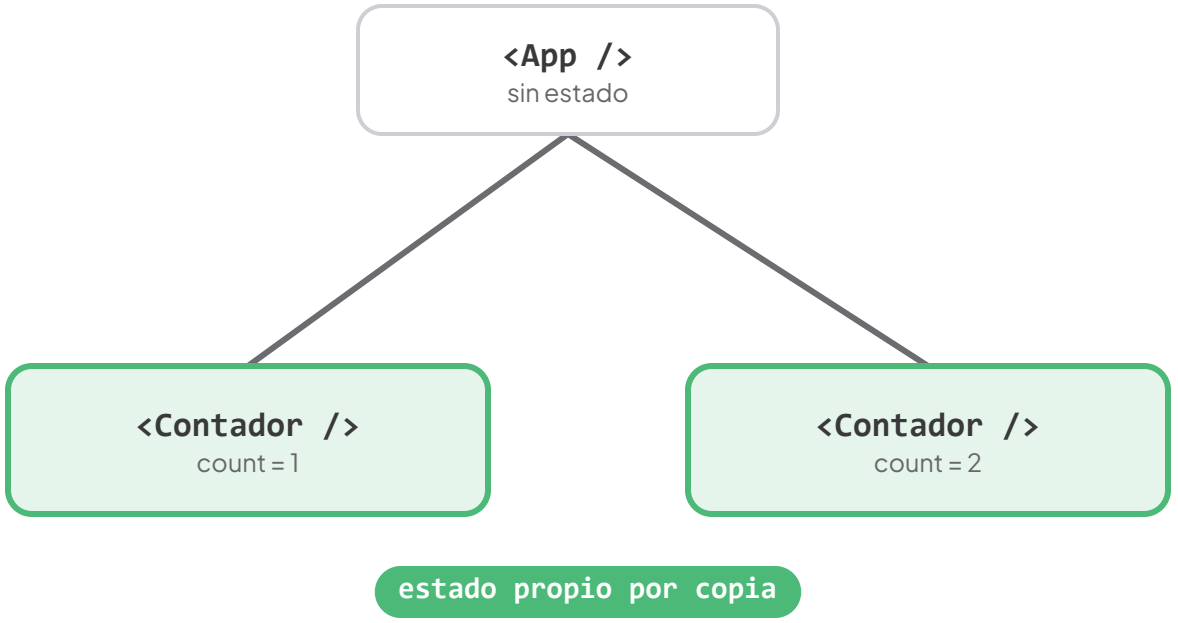




React compara el resultado nuevo con el anterior y actualiza **solo el texto que cambió**. El resto del DOM se queda como estaba.

Cada copia tiene su propio estado





Pulsa dos veces el segundo botón: su `count` sube a 2 y el del primero sigue en 1. React asocia el estado al **lugar que ocupa** cada copia en el árbol.

El estado es una foto

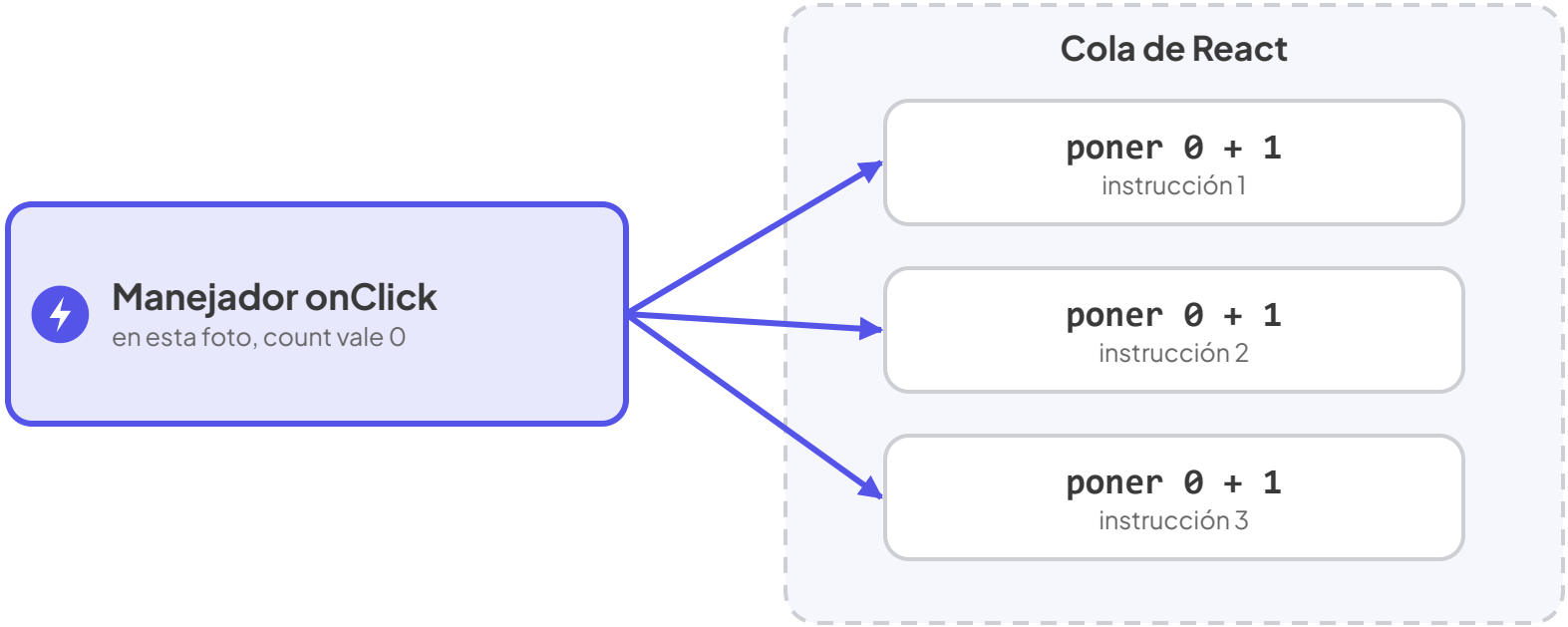
Contador.jsx

```jsx
<button onClick={() => {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}}>
  +3
</button>
```

Cada render tiene su foto
En este render, `count` vale 0 en todo el manejador.

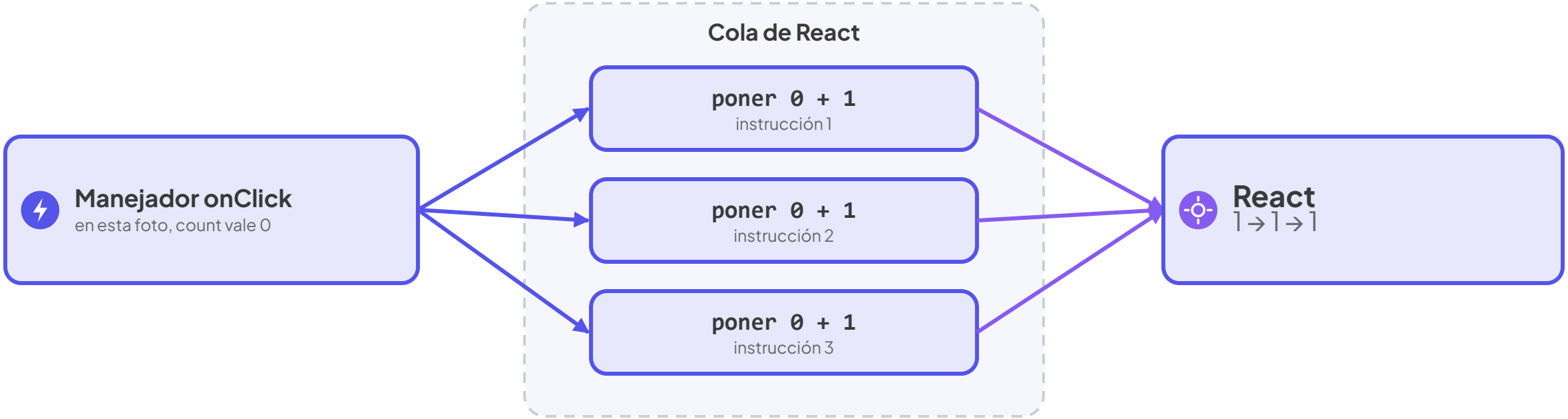
¿Cuánto vale count después del clic?
¿3? Veamos qué hace React.

Cada setCount deja una instrucción en una cola



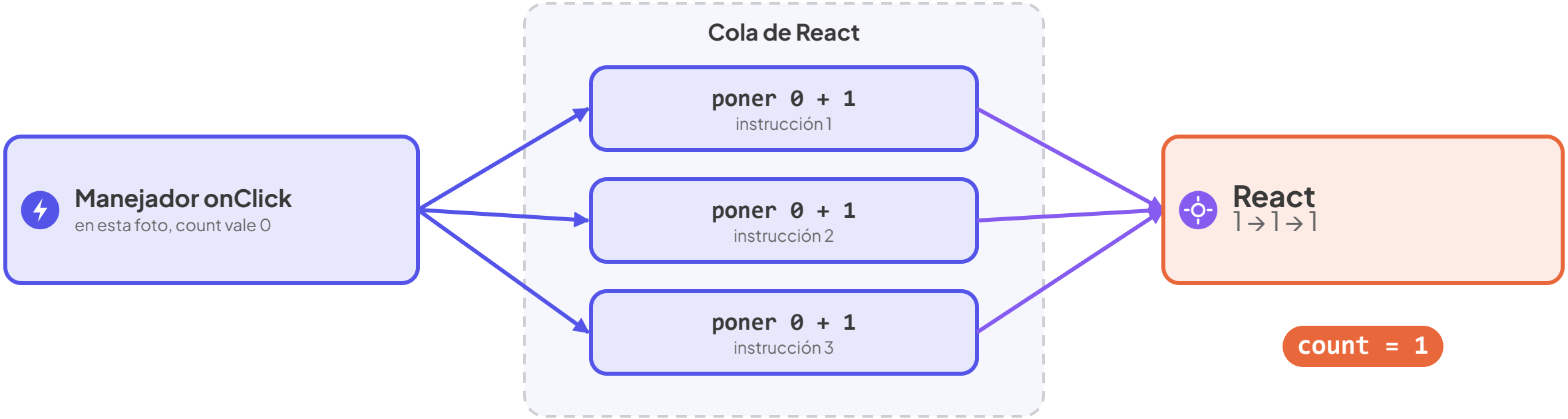
Las tres llamadas usan la misma foto (`count = 0`), así que las tres piden lo mismo: **«poner 0 + 1»**.

React procesa la cola en orden



Aplica las tres instrucciones una tras otra: **1, 1, 1**. Todas dicen lo mismo.

El resultado es 1, no 3



El siguiente render recibe `count = 1`. Si el valor nuevo depende del anterior, hay que decírselo a React de otra forma.

Universidad ICESI

Una función actualizadora usa el valor anterior

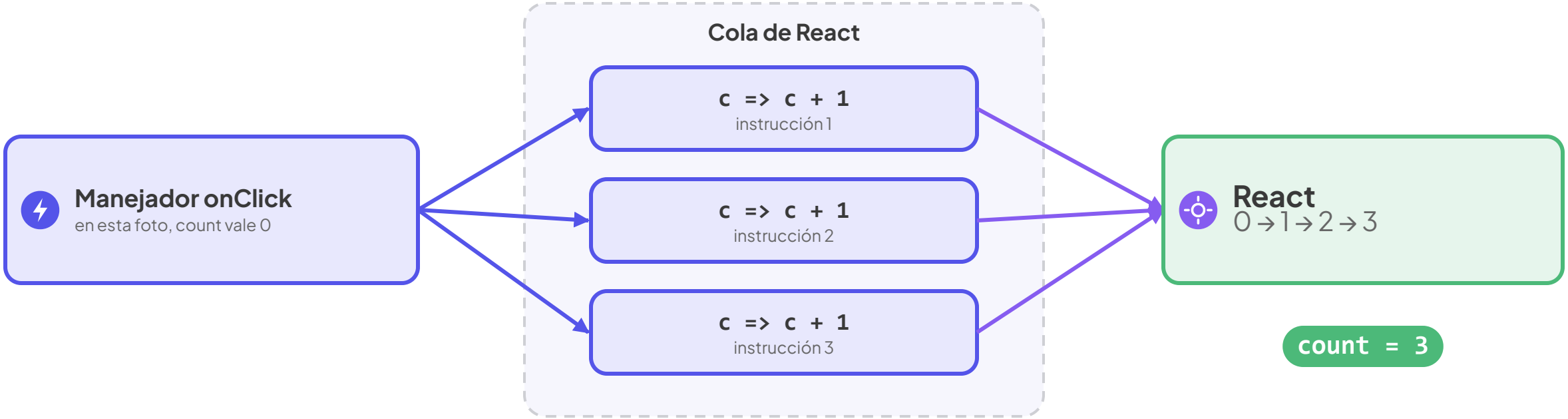
Contador.jsx

```jsx
setCount((c)=> c + 1); //0→1
setCount((c)=> c + 1); //1→2
setCount((c)=> c + 1); //2→3
```

c => c + 1
React la llama con el **resultado de la anterior**.

Cuándo usarla
Cuando el valor nuevo se calcula a partir del anterior.

Con la forma funcional, las tres instrucciones se encadenan



Cada función recibe lo que dejó la anterior: **0 → 1 → 2 → 3**. Después del clic, `count` vale 3.

No mutes el estado: reemplázalo

Mutar (no funciona)

```
// Mal: React no se entera
user.name = "Ana";
setUser(user);

items.push(nuevo);
setItems(items);
```

Reemplazar

```
// Bien: un valor nuevo
setUser({ ...user, name: "Ana" });

setItems([...items, nuevo]);

setItems(items.filter((i) => i.id !== id));
```

> React compara por **referencia**: solo se entera del cambio si le pasas un objeto o arreglo **nuevo**.

Universidad ICESI

Reglas de los hooks

React los identifica por el orden de las llamadas

Cada `useState` ocupa el siguiente lugar en la memoria del componente.

Solo en el nivel superior

No los llames dentro de `if`, bucles ni funciones anidadas: el orden debe ser el mismo en cada render.

Solo en componentes o en otros hooks

No en funciones normales de JavaScript.

Universidad ICESI

2. useEffect

Sincronizar con el mundo exterior

Renderizar es puro; los efectos van después

DURANTE EL RENDER

Solo calcular qué dibujar.

- Sin tocar nada fuera del componente.
- Mismas props y estado, mismo resultado.

UN EFECTO (USEEFFECT)

Hacer algo con el exterior.

- Se ejecuta **después** de actualizar el DOM.
- Conectarse a un chat, cambiar el título, iniciar un temporizador.

Universidad ICESI

Un efecto que se conecta a una sala

Chat.jsx

```jsx
function Chat({roomId}) {
  const [message, setMessage] = useState("");

  useEffect(()=> {
    const conn = connect(roomId);
    return ()=> conn.disconnect();
  }, [roomId]);

  //...
}
```

Tres partes
Setup, limpieza y dependencias.

Ejemplo
Conectarse a la sala `roomId` mientras el componente esté en pantalla.

Setup: lo que hace el efecto

Chat.jsx

```jsx
function Chat({roomId}) {
  const [message, setMessage] = useState("");

  useEffect(() => {
    const conn = connect(roomId);
    return () => conn.disconnect();
  }, [roomId]);

  //...
}
```

El cuerpo del efecto
Corre después de dibujar, la primera vez y cada vez que cambia una dependencia.

Aquí
Abre la conexión a la sala.

Limpieza: deshacer lo que hizo el setup

Chat.jsx

```jsx
function Chat({roomId}) {
  const [message, setMessage] = useState("");

  useEffect(()=> {
    const conn = connect(roomId);
    return ()=> conn.disconnect();
  }, [roomId]);

  //...
}
```

La función que devuelve
Corre antes de repetir el efecto y al desmontar el componente.

Aquí
Cierra la conexión para no dejar nada abierto.

Dependencias: cuándo debe repetirse

Chat.jsx

```jsx
function Chat({roomId}) {
  const [message, setMessage] = useState("");

  useEffect(()=> {
    const conn = connect(roomId);
    return ()=> conn.disconnect();
  }, [roomId]);

  //...
}
```

El arreglo
Los valores reactivos (props y estado) que el efecto lee.

React compara
Si alguno cambió desde el render anterior, repite el efecto.

El arreglo decide cuándo corre el efecto

[roomId]

Al montar y **cada vez que roomId cambia**. Es el caso correcto.

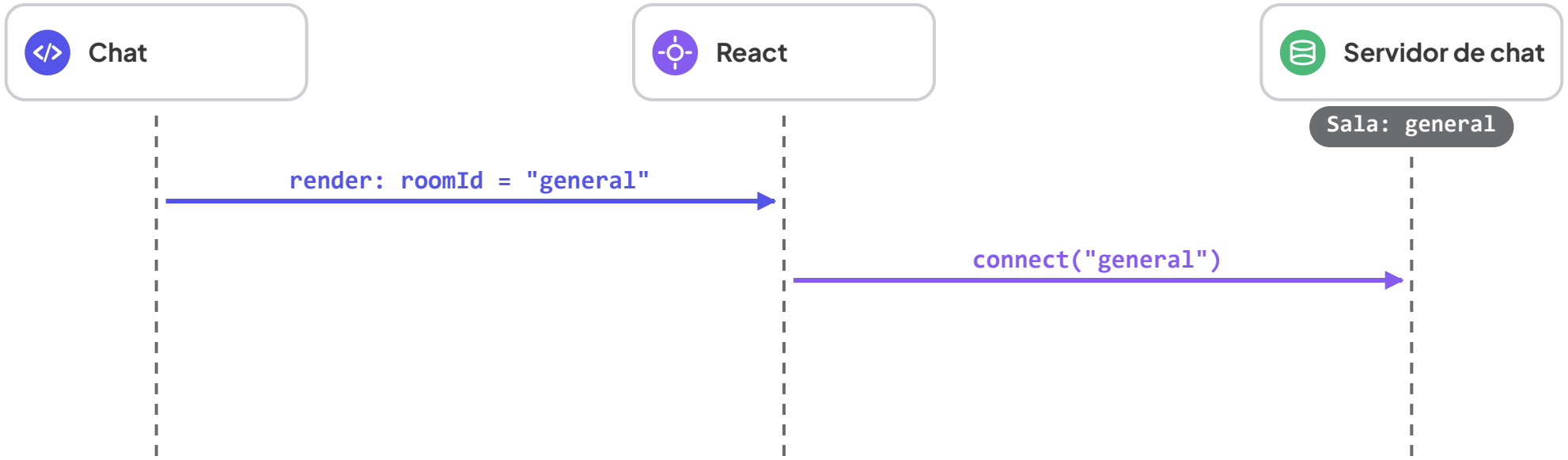
[]

Solo al montar. Si el efecto usa `roomId`, **no se entera** de que cambió.

Sin arreglo

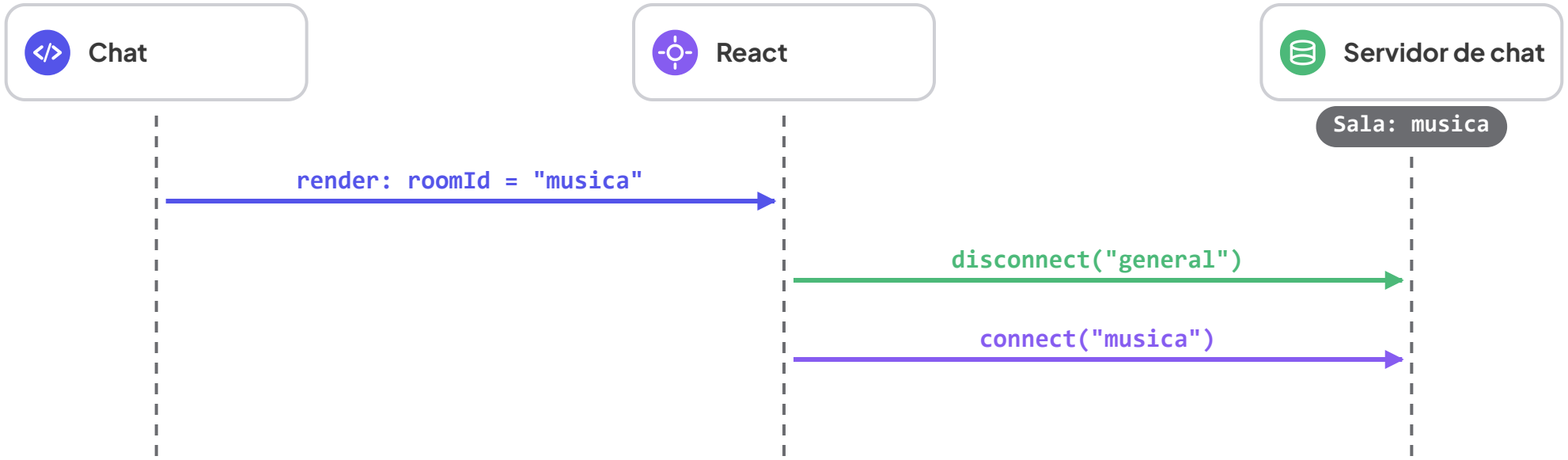
Después de **cada render**, también cuando cambia cualquier otro estado.

Al montar, el efecto se ejecuta



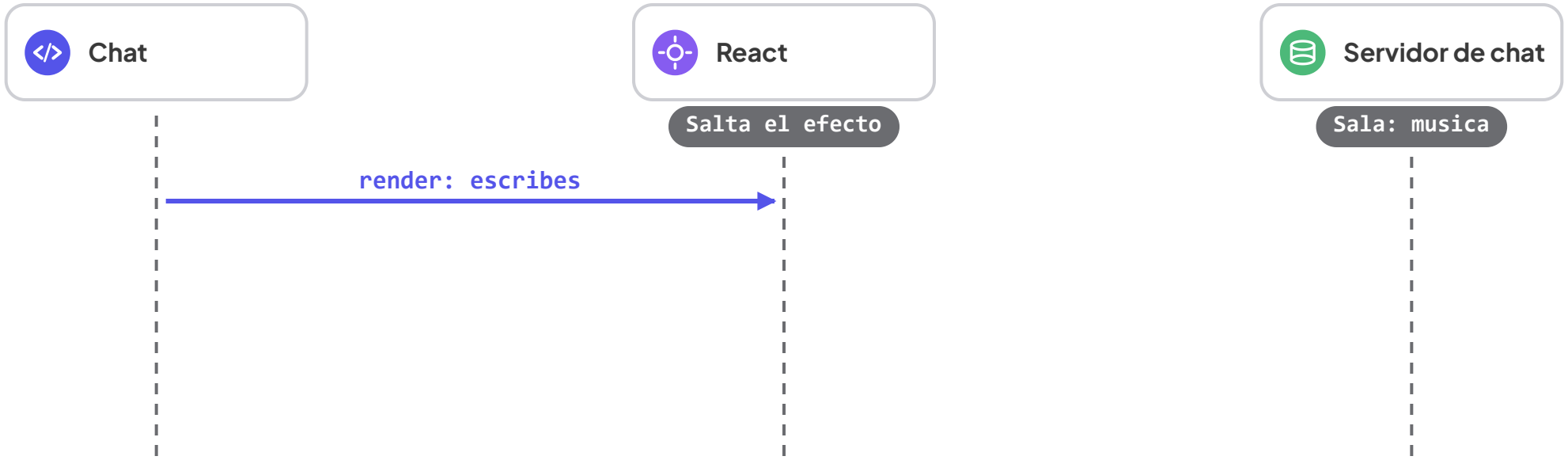
El componente se dibuja y, **ya en pantalla**, React ejecuta el efecto: se conecta a la sala «general». Pasa con cualquier arreglo de dependencias.

Cambia roomId: limpiar y volver a conectar



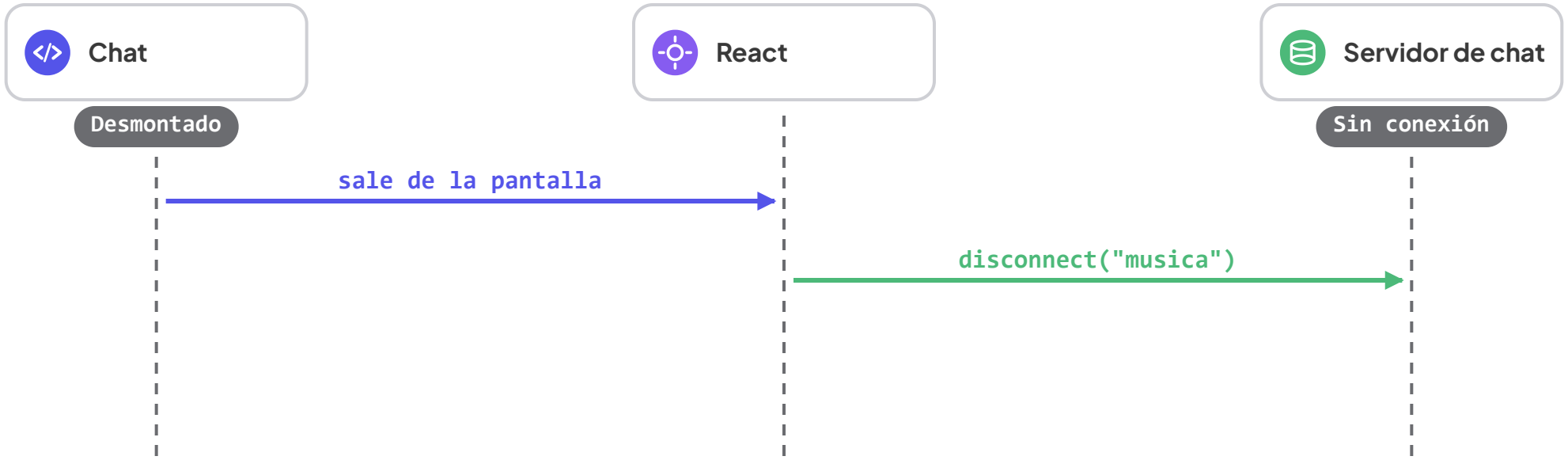
Con `[roomId]`, React primero ejecuta la **limpieza** del efecto anterior y después el efecto nuevo con el valor actual.

Escribes un mensaje: el efecto no corre



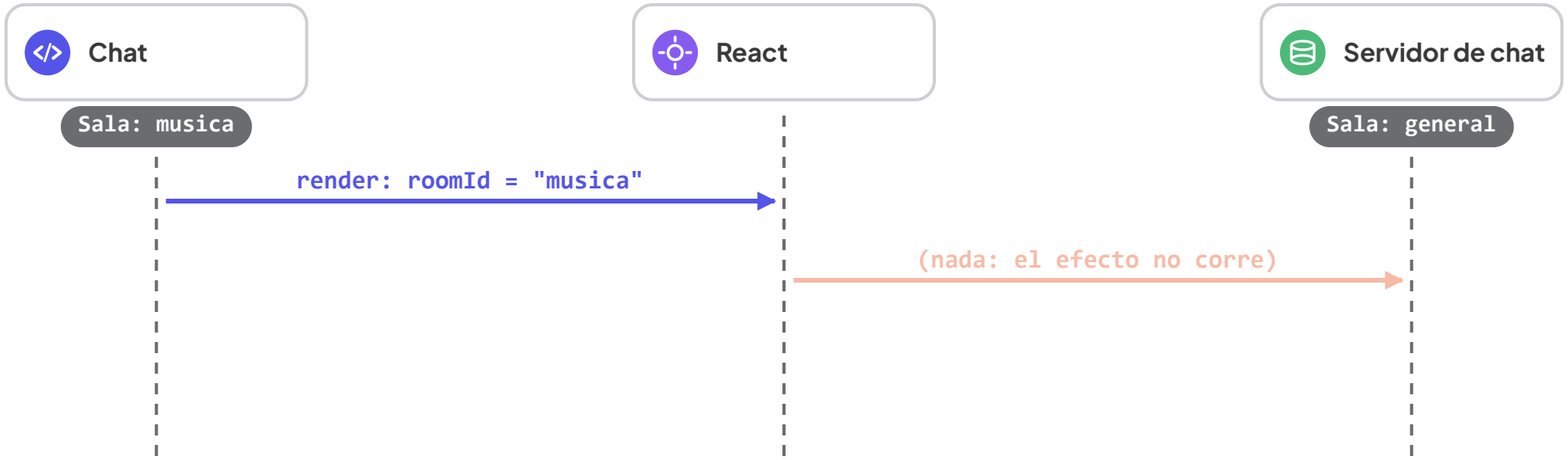
Escribir cambia el estado y vuelve a ejecutar `Chat`, pero `roomId` no cambió: React se salta el efecto y **conserva la conexión**.

Al desmontar, solo la limpieza



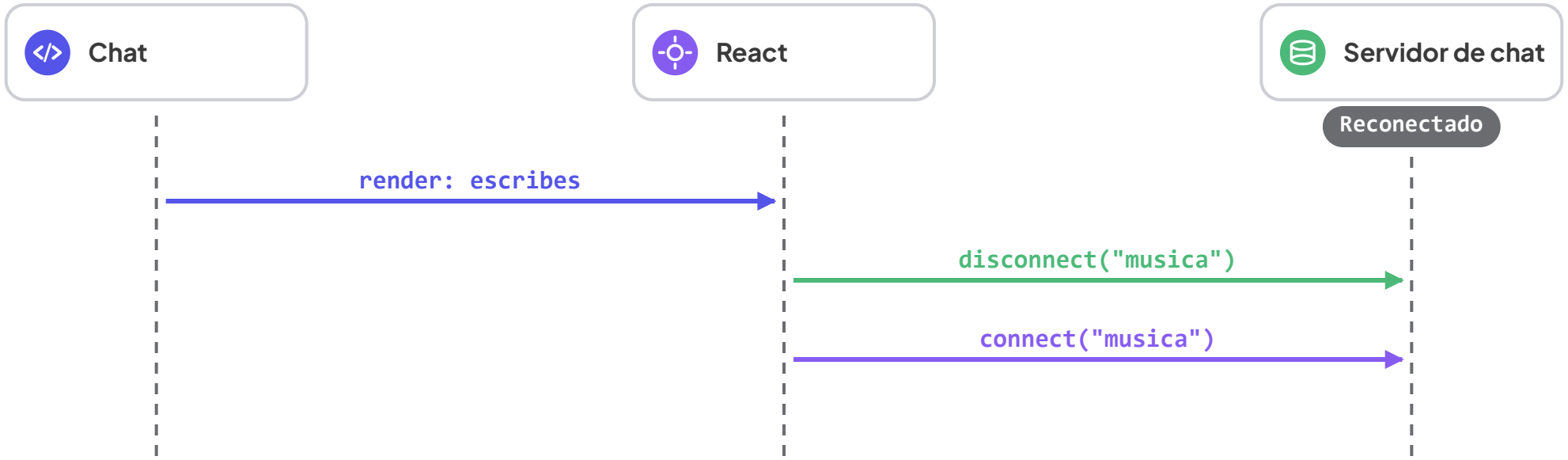
Al salir de la pantalla React ejecuta la limpieza por última vez: la conexión se cierra y **no queda nada abierto**.

Con [] el efecto no se entera del cambio



Con `[]` el efecto solo corre al montar: la pantalla dice «musica», pero **el chat sigue en «general»**. Es el error más común.

Sin arreglo, se reconecta en cada tecla



Sin arreglo el efecto corre después de **cada render**: escribir un mensaje desconecta y vuelve a conectar el chat en cada tecla.

Universidad ICESI



¿Por qué corre dos veces?

En desarrollo, con `<StrictMode>` (la plantilla de Vite lo trae), React monta el componente, **ejecuta la limpieza** y lo monta otra vez.

Lo hace a propósito: comprueba que tu limpieza deshace bien lo que hizo el setup. **En producción corre una sola vez.**

Universidad ICESI

Un efecto para pedir datos

Usuario.jsx

```jsx
function Usuario({userId}) {
  const [user, setUser] = useState(null);

  useEffect(()=> {
    let ignore = false;

    fetch(`https://api.example.com/users/${userId}`)
      .then((res)=> res.json())
      .then((data)=> {
        if (!ignore) setUser(data);
      });

    return ()=> { ignore = true; };
  }, [userId]);

  if (!user) return <p>Cargando...</p>;
  return <h1>{user.name}</h1>;
}
```

Pedir datos es un efecto
No depende del render, sino de algo externo: el servidor.

La limpieza evita un error
Si `userId` cambia mientras viaja la primera petición, esa respuesta se descarta.

¿Siempre necesito un efecto?

SÍ, PARA SINCRONIZAR

Con algo externo a React.

- Conectarse a un servidor.
- Suscribirse a eventos del navegador.
- Iniciar y limpiar un temporizador.

NO, PARA ESTO

Hazlo sin efecto.

- Un valor calculado a partir de props o estado: **calcúlalo durante el render**.
- La respuesta a un clic: **ponlo en el manejador del evento**.

Universidad ICESI

3. Custom hooks

Reutilizar la lógica

Universidad ICESI

Cuando varios componentes repiten lo mismo

SIN CUSTOM HOOK

Estado y efecto copiados.

- Cada componente repite el mismo `useState` y el mismo `useEffect`.
- Un arreglo hay que hacerlo en todos los lugares.

CON UN CUSTOM HOOK

Una función con la lógica.

- Su nombre empieza con `use`.
- Cada componente la llama en una línea.

Una función que empieza con use

useOnlineStatus.js

```
function useOnlineStatus() {
  const [online, setOnline] = useState(true);
  useEffect(()=> {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    addEventListener("online", on);
    addEventListener("offline", off);
    return ()=> {
      removeEventListener("online", on);
      removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

Es una función normal

El prefijo `use` avisa a React y al linter que dentro hay hooks.

Puede guardar estado

useOnlineStatus.js

```
function useOnlineStatus() {
  const [online, setOnline] = useState(true);
  useEffect(()=> {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    addEventListener("online", on);
    addEventListener("offline", off);
    return ()=> {
      removeEventListener("online", on);
      removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

useState
Ese estado **pertenece al componente** que use el hook, no al hook.

Puede sincronizarse con algo externo

useOnlineStatus.js

```
function useOnlineStatus() {
  const [online, setOnline] = useState(true);
  useEffect(()=> {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    addEventListener("online", on);
    addEventListener("offline", off);
    return ()=> {
      removeEventListener("online", on);
      removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

useEffect
Se suscribe a los eventos de red del navegador al montar, una sola vez.

Y limpiar lo que abrió

useOnlineStatus.js

```
function useOnlineStatus() {
  const [online, setOnline] = useState(true);
  useEffect(()=> {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    addEventListener("online", on);
    addEventListener("offline", off);
    return ()=> {
      removeEventListener("online", on);
      removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

Limpieza
Al desmontar quita los eventos: nada sigue llamando a `setOnline`.

Universidad ICESI

Devuelve lo que necesitan los componentes

useOnlineStatus.js

```
function useOnlineStatus() {
  const [online, setOnline] = useState(true);
  useEffect(()=> {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    addEventListener("online", on);
    addEventListener("offline", off);
    return ()=> {
      removeEventListener("online", on);
      removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

return
Un valor, un arreglo, un objeto o funciones. Quien lo usa solo ve el resultado.

Usarlo cabe en una línea

StatusBar.jsx

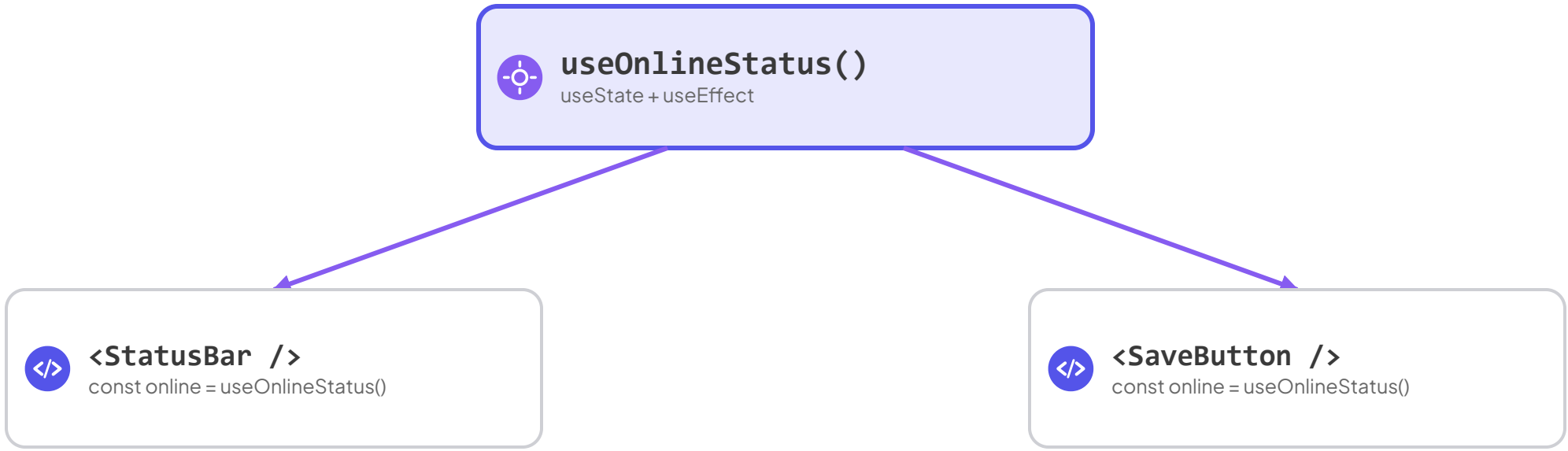
```jsx
function StatusBar() {
  const online = useOnlineStatus();
  return <h1>{online ? "Conectado" : "Sin conexión"}</h1>;
}

function SaveButton() {
  const online = useOnlineStatus();
  return <button disabled={!online}>Guardar</button>;
}
```

Una línea
Reemplaza todo el estado y el efecto.

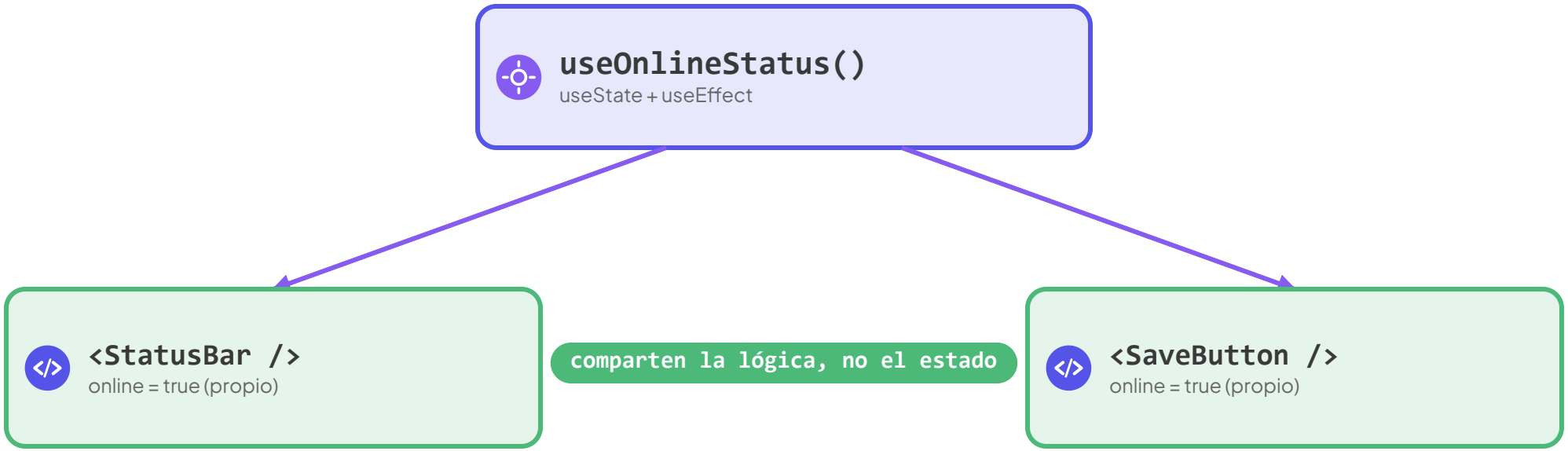
Dos componentes
La barra de estado y el botón de guardar usan el mismo hook.

Un hook, muchos componentes



Cada componente llama a `useOnlineStatus()`. El código del hook se escribe **una sola vez**.

Se comparte la lógica, no el estado



Cada llamada crea su propio `online`: dos llamadas a `useState`, **dos estados** independientes.

Universidad icesi

Qué debes llevarte de los custom hooks

El nombre importa
Empieza con `use` para que React y el linter apliquen las reglas de los hooks.

Un hook, una tarea
`useOnlineStatus`, `useWindowWidth`, `useFetch`. Si hace demasiado, divídelo.

Dónde vive
En la carpeta `hooks/` del proyecto: `src/hooks/useOnlineStatus.js`.

Lógica, no estado
Dos componentes que usan el mismo hook **no comparten** sus valores.

Universidad ICESI

4. React Router

Varias páginas en una SPA

Los usuarios esperan varias páginas

SITIO CLÁSICO

Un archivo por URL.

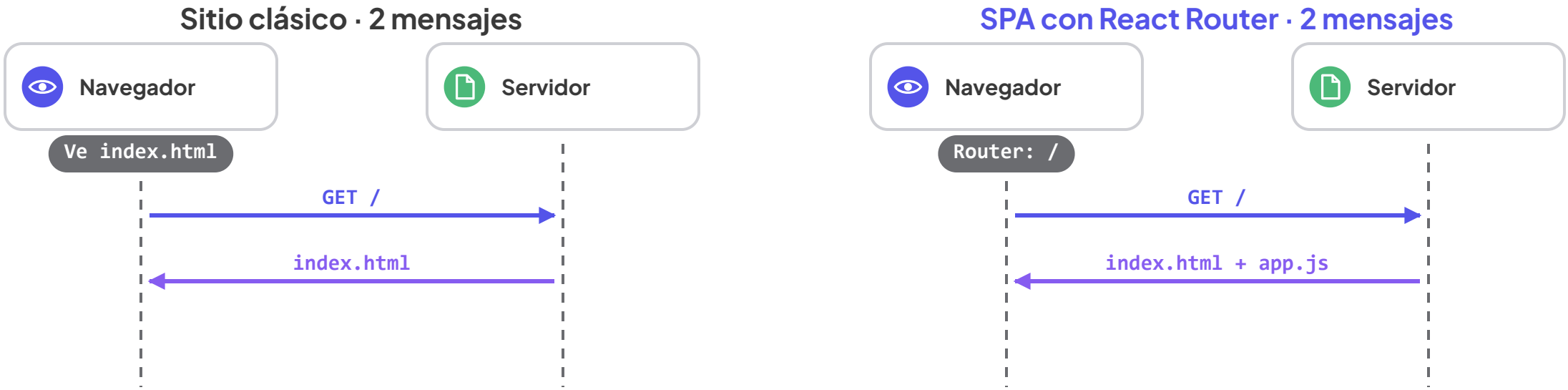
- `/productos` pide `productos.html`.
- Cada clic recarga la página completa.

SPA (SINGLE PAGE APPLICATION)

Un solo `index.html`.

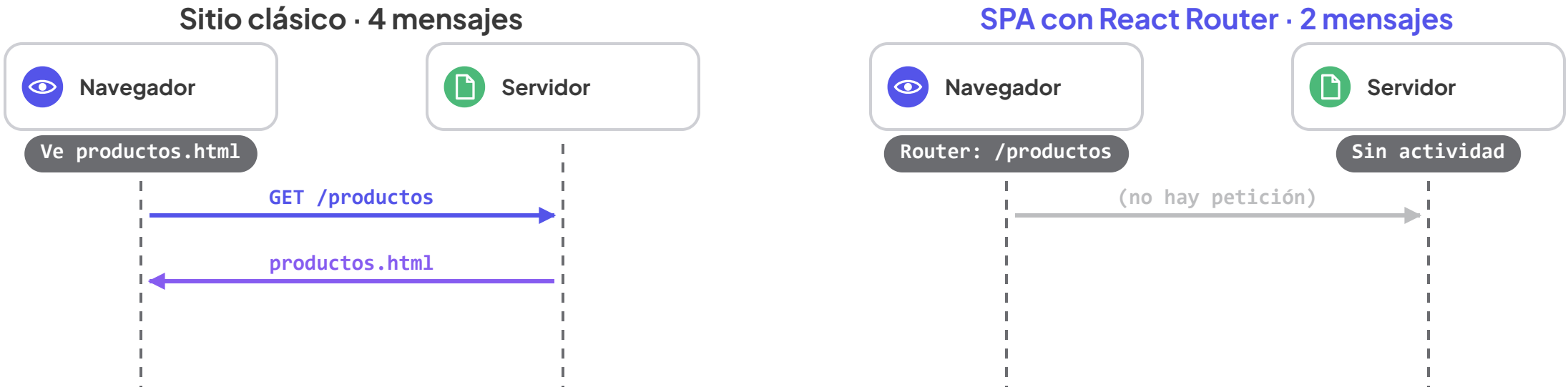
- El JavaScript arma lo que ves.
- Pero seguimos queriendo URL que se compartan y un botón Atrás que funcione.

Entrar a la página principal



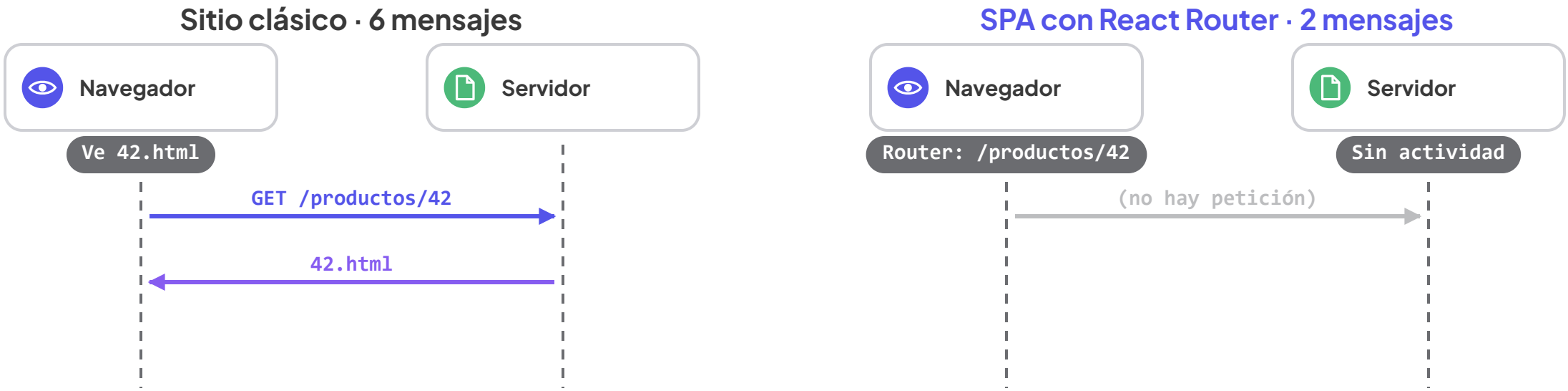
En los dos casos el servidor responde con un HTML. La SPA además trae el JavaScript de la aplicación: **ahí vive React Router**.

Clic en «Productos» (/productos)



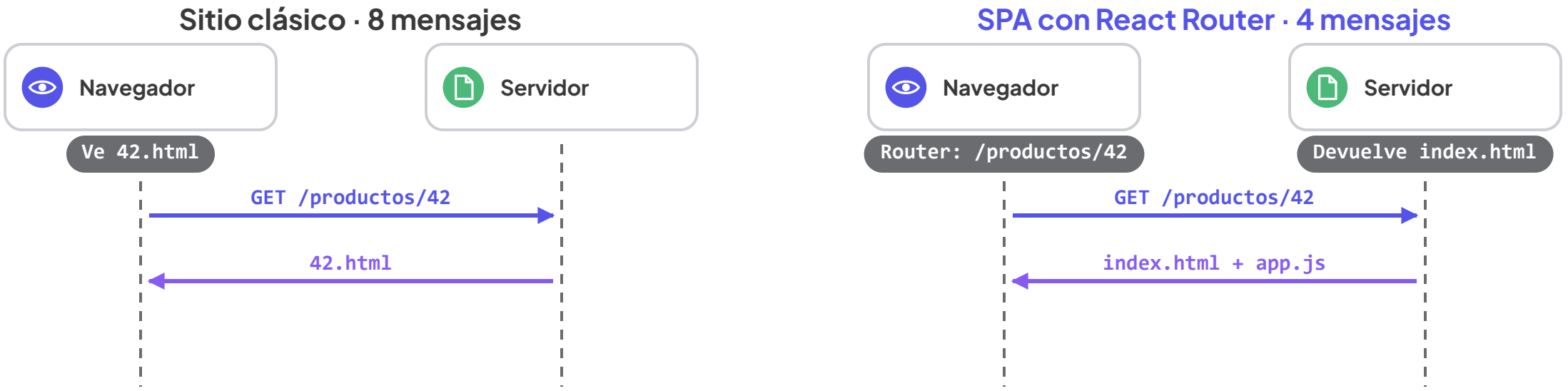
El sitio clásico pide otro archivo y recarga la página. La SPA **no pide nada**: el router lee la URL, elige el componente y React cambia el DOM.

Clic en un producto (/productos/42)



Un sitio clásico necesitaría un archivo por producto. En la SPA **una sola ruta** con parámetro (`productos/:id`) sirve para todos.

Recargar la página (F5)



Al recargar, el navegador sí le pregunta al servidor. Como solo existe `index.html`, el servidor debe **devolverlo para cualquier ruta**; después el router arma la página.

Universidad ICESI

Qué debes llevarte de la SPA

React Router no busca archivos
La URL es un dato: el router la compara con una lista de rutas y decide qué componentes dibujar.

Navegar no recarga la página
Cambia la URL con la API `history` y React actualiza solo lo que cambió.

La URL sigue siendo útil
Se copia, se guarda en favoritos y funciona con Atrás y Adelante.

Al desplegar
Configura el servidor para devolver `index.html` en cualquier ruta (`try_files` , `rewrites`).

Universidad ICESI

Tres modos de usar React Router

Declarative

Con componentes: `<Routes>` y `<Route>`. Lo más simple, sin carga de datos.

Data (el del curso)

Con objetos: `createBrowserRouter`. Rutas, `loader`s y manejo de errores.

Framework

Con archivos y un plugin de Vite. Aplicaciones completas con SSR.

Instalar React Router

Terminal

```
npm install react-router
```

Desde la versión 7 todo se importa de `react-router`. En tutoriales viejos verás el paquete `react-router-dom`.

Un solo paquete
Dentro de tu proyecto de Vite.

Modo Data
Es el que usaremos: rutas como objetos.

Crear el router y entregarlo a RouterProvider

src/main.jsx

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { createBrowserRouter } from "react-router";
import { RouterProvider } from "react-router/dom";

const router = createBrowserRouter([
  { path: "/", Component: () => <h1>Hola desde "/"</h1> },
]);

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <RouterProvider router={router} />
  </StrictMode>
);
```

createBrowserRouter
Recibe la lista de rutas y crea el router.

RouterProvider
Reemplaza a `<App />`: decide qué se dibuja según la URL.

Una sola vez
Crea el router **fuera** de cualquier componente.

5. Rutas

Páginas y subpáginas

Universidad ICESI

Cada ruta es un objeto

src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

path
El segmento de URL que la ruta atiende.

Component
Lo que se dibuja cuando la URL coincide. La raíz hace de **layout**: lo fijo de la página.

children: rutas dentro de rutas

src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

children
Las rutas hijas se dibujan en el hueco `<Outlet />` de su padre.

El path se suma
`"/"` + `"productos"` + `":id"`.

index: la hija por defecto

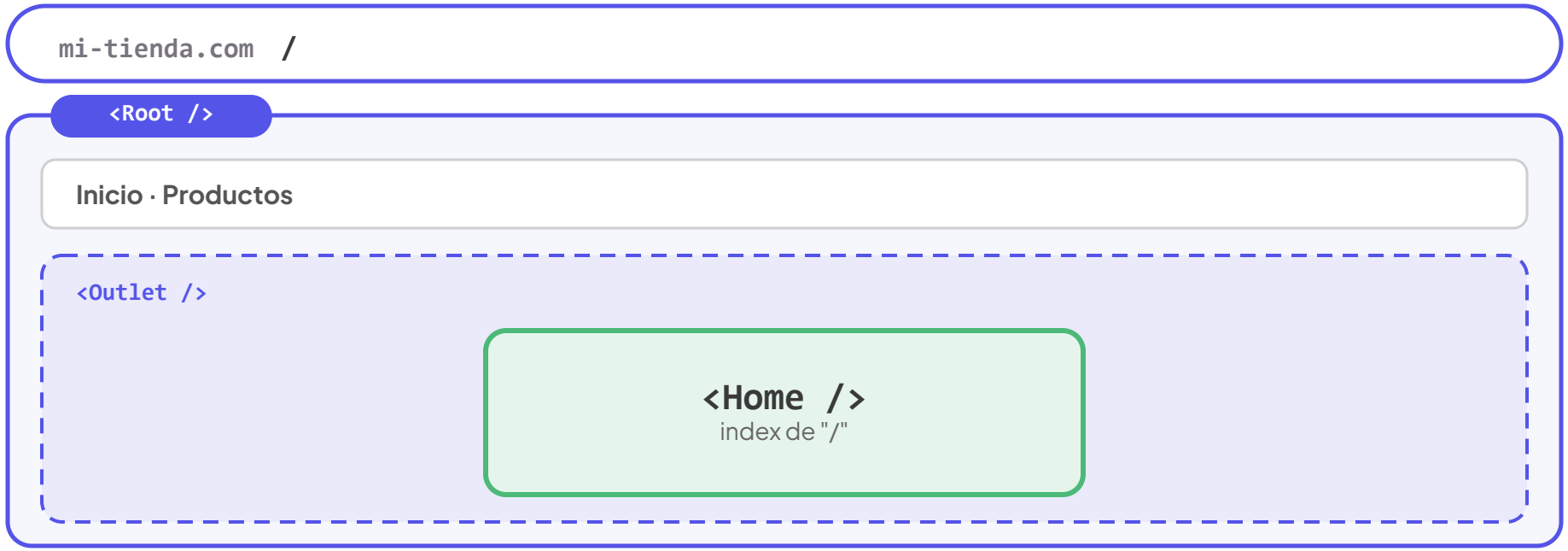
src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

index: true
Se dibuja cuando la URL es **exactamente** la del padre.

Sin path y sin hijos
Una ruta index no lleva `path` ni puede tener hijas.

En «/» el Outlet muestra la ruta index



La URL es exactamente la de la raíz: `Root` se dibuja siempre y en su `Outlet` aparece **Home**, la ruta marcada con `index`.

Un prefijo agrupa subpáginas

src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

path sin Component
Agrupa a sus hijas bajo un prefijo de URL, sin dibujar nada propio.

Aquí
Todas las hijas cuelgan de `/productos`.

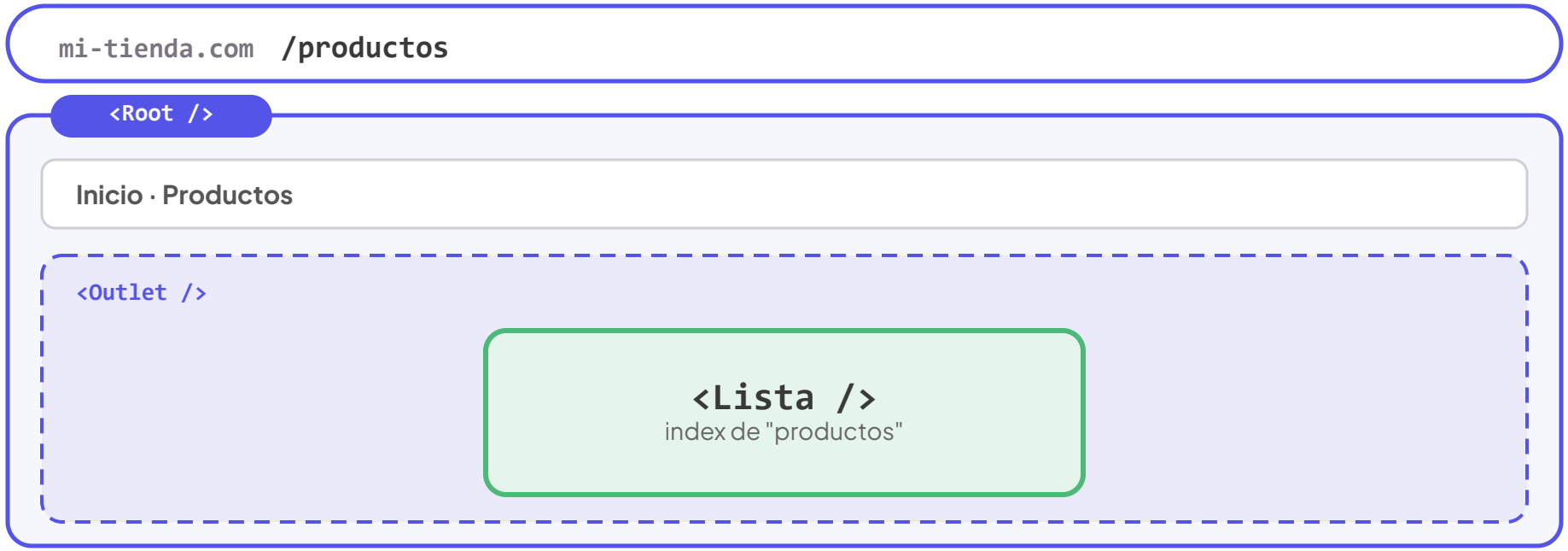
index dentro del prefijo

src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

La lista
Se dibuja cuando la URL es justo `/productos`.

En «/productos» el Outlet muestra la lista



El layout sigue siendo el mismo: solo **cambia lo que hay dentro del Outlet**. `Root` no se vuelve a crear.

:id coincide con cualquier valor

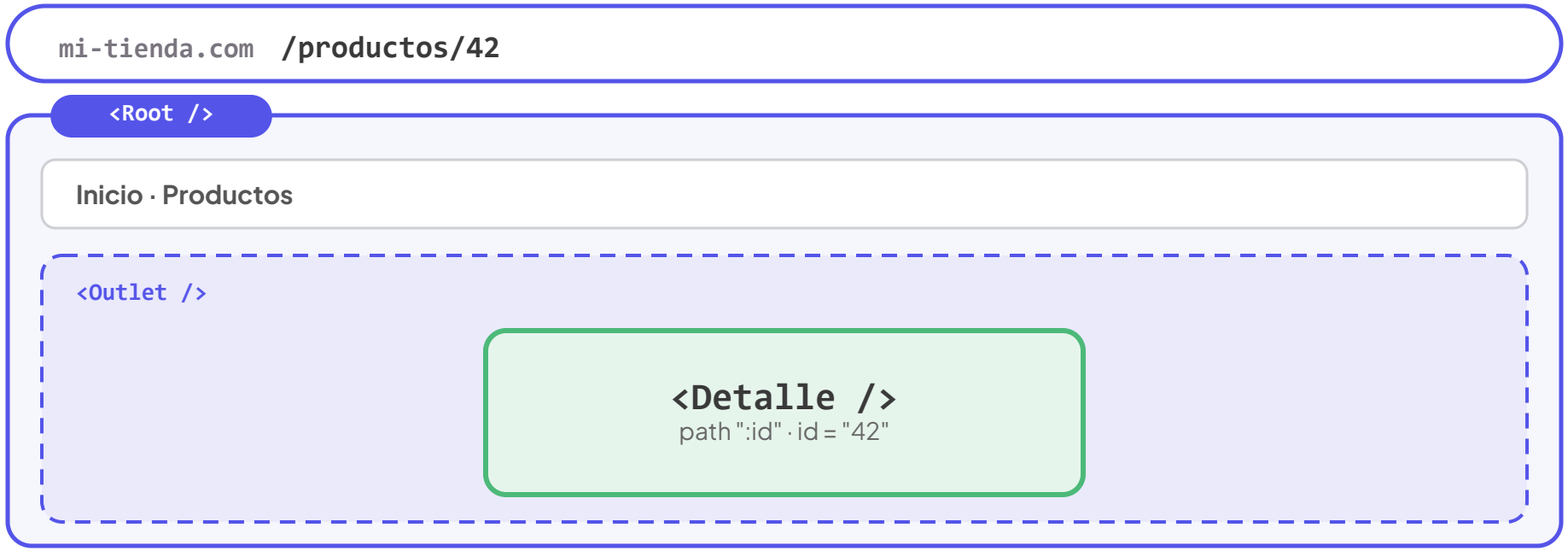
src/router.jsx

```jsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
      { index: true, Component: Home },
      {
        path: "productos",
        children: [
          { index: true, Component: Lista },
          { path: ":id", Component: Detalle },
        ],
      },
    ],
  },
]);
```

Segmento dinámico
Un segmento que empieza con `:` coincide con **cualquier valor**.

Una ruta para todos
Sirve para `/productos/42`, `/productos/7`…

En «/productos/42» el Outlet muestra el detalle



Una sola ruta atiende todos los productos. El valor del segmento queda disponible con `useParams()`.

El layout deja un hueco con Outlet

src/pages/Root.jsx

```jsx
import { Outlet, NavLink } from "react-router";

export default function Root() {
  return (
    <>
      <nav>
        <NavLink to="/">Inicio</NavLink>
        <NavLink to="/productos">Productos</NavLink>
      </nav>

      <Outlet />
    </>
  );
}
```

Lo fijo
La barra de navegación se dibuja siempre.

<Outlet />
Aquí se dibuja la ruta hija que coincide con la URL.

Enlaces que no recargan la página

<LINK TO="/PRODUCTOS">

Cambia la URL y le avisa al router.

- Reemplaza a `<a href>`, que pide la página al servidor y recarga todo.

<NAVLINK TO="/PRODUCTOS">

Igual, pero sabe si es la ruta actual.

- Le agrega la clase `active`.
- Ideal para los menús.

Una página para lo que no existe

src/router.jsx

```
{ path: "*", Component: NotFound },
```

El comodín *
Coincide con cualquier URL que no coincidió con otra ruta.

Al final de la lista
Dibuja la página 404 de tu aplicación.

Universidad ICESI

Las piezas de una ruta

path

El segmento de URL que atiende. El del padre se suma al de los hijos.

Component

Lo que se dibuja cuando la ruta coincide.

children

Rutas anidadas: se dibujan en el `Outlet` del padre.

index: true

La hija por defecto, en la URL exacta del padre.

Sin path / sin Component

Sin `path`: un layout. Sin `Component`: un prefijo.

:id y *

`:id` coincide con cualquier valor; `*` con cualquier URL restante (404).

Universidad ICESI

6. Navegar

La pila del historial

useNavigate: navegar desde el código

src/pages/Detalle.jsx

```jsx
import { useNavigate } from "react-router";

export default function Detalle() {
  const navigate = useNavigate();

  return (
    <>
      <button onClick={() => navigate("/carrito")}>
        Agregar al carrito
      </button>
      <button onClick={() => navigate(-1)}>Volver</button>
    </>
  );
}
```

navigate("/carrito")
Va a esa ruta.

navigate(-1)
Vuelve a la página anterior.

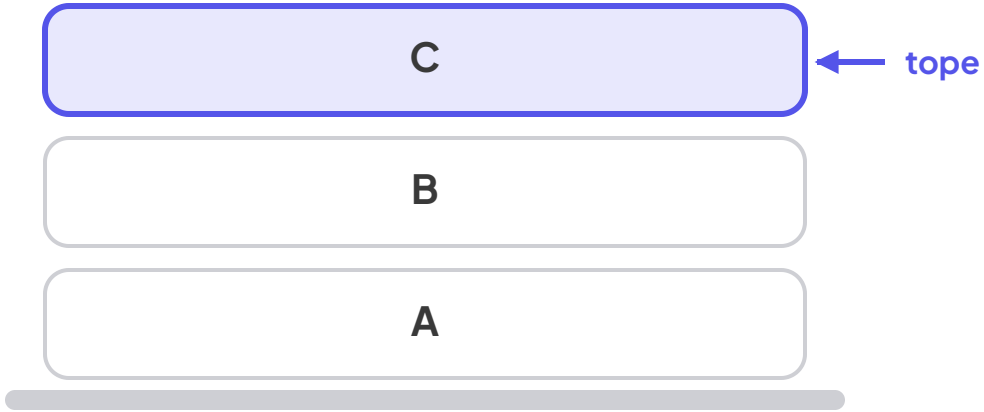
¿Qué hacen por dentro?
Hay que entender dónde guarda el navegador las páginas que visitas.

Una pila solo se toca por un extremo



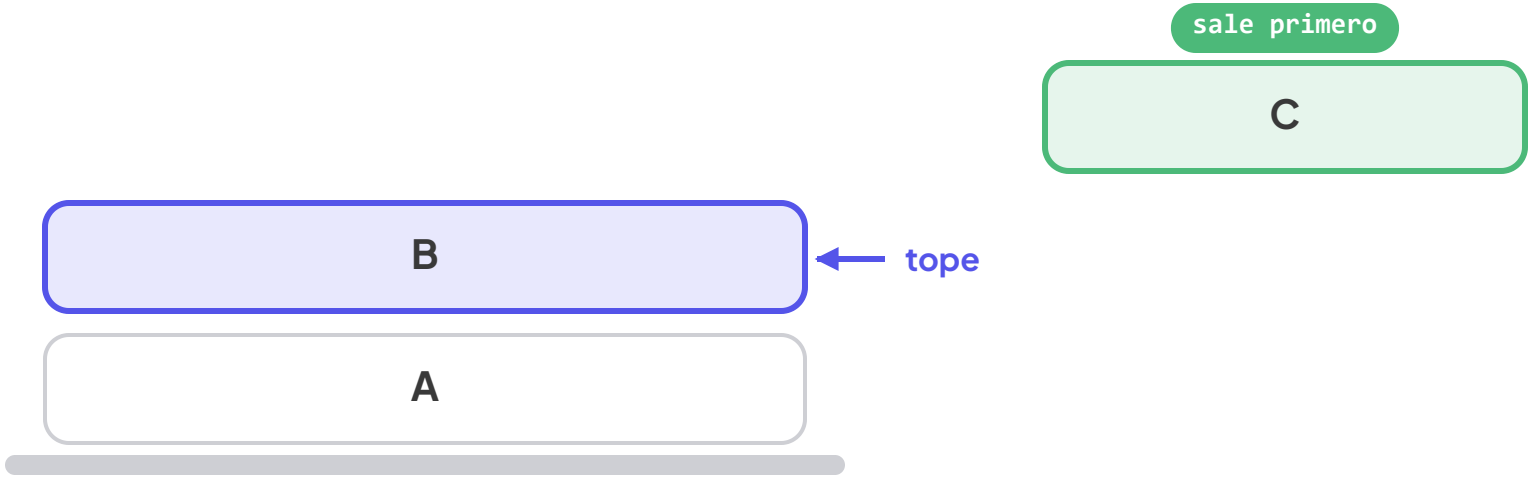
Agregas un elemento por arriba: `push`. El de arriba se llama **tope**.

Cada push apila otro encima



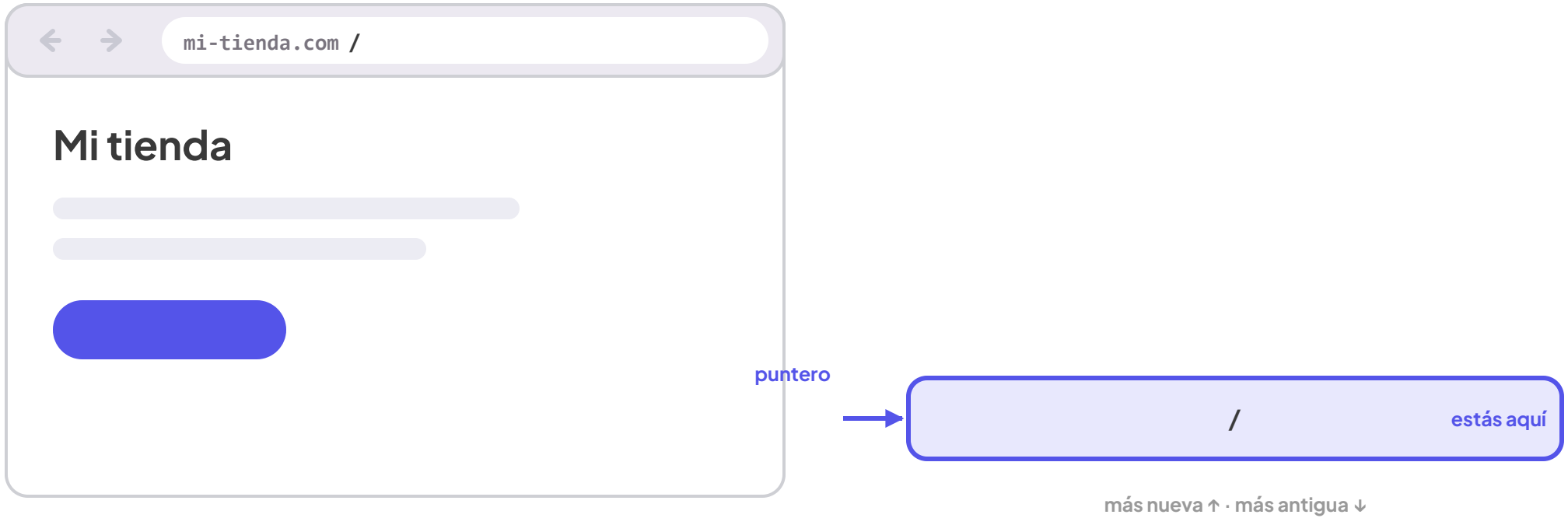
Ahora hay tres elementos. Solo puedes tocar el **tope**: C. A y B esperan debajo.

pop saca el del tope



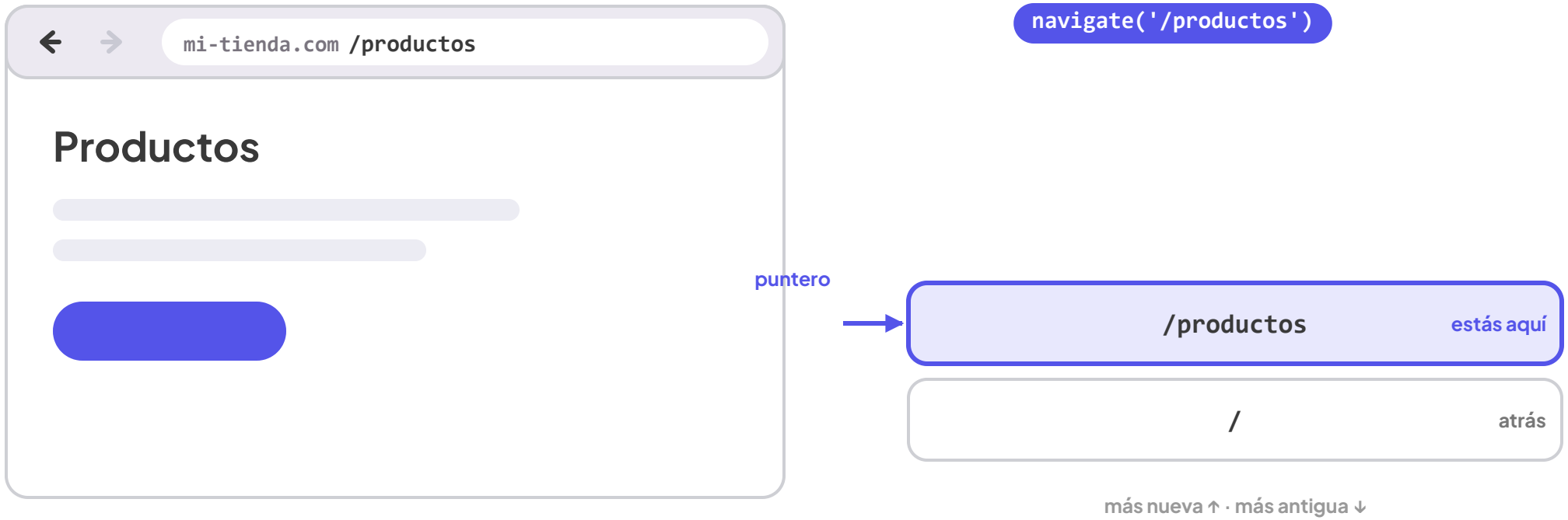
El último en entrar es el primero en salir (LIFO), como una pila de platos. Así guarda el navegador las páginas que visitas.

El historial arranca con una entrada



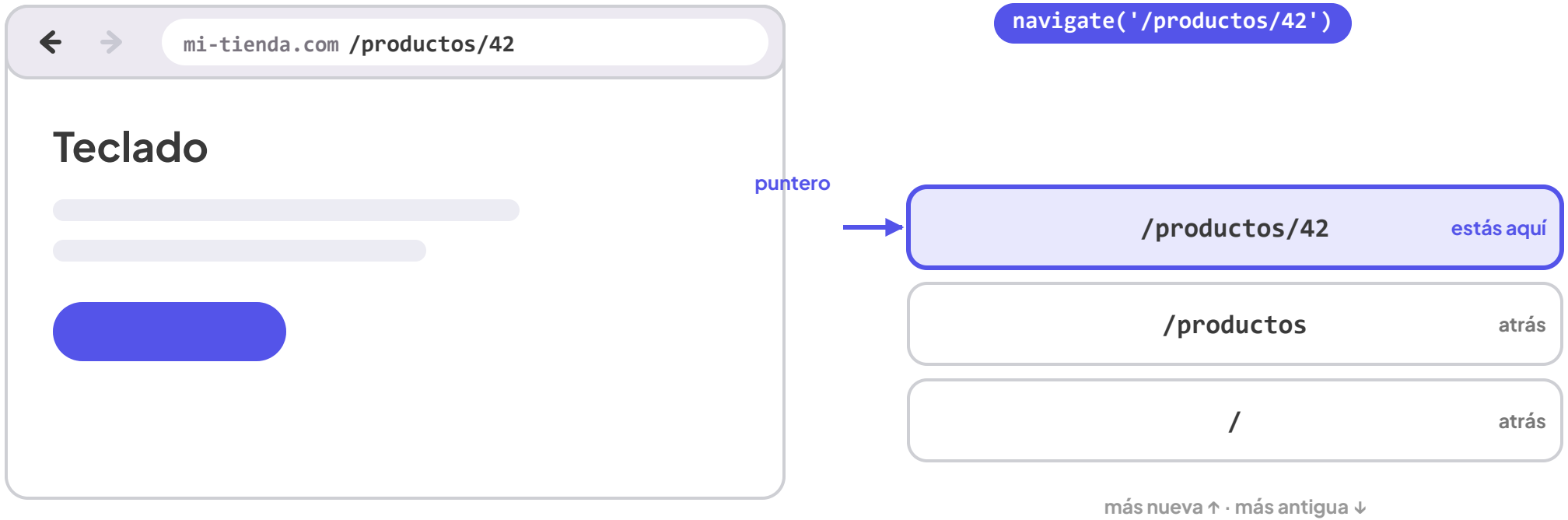
Al abrir la app el navegador anota la primera página. Con una sola entrada **no hay a dónde volver**: la flecha Atrás está apagada.

navigate apila una entrada nueva



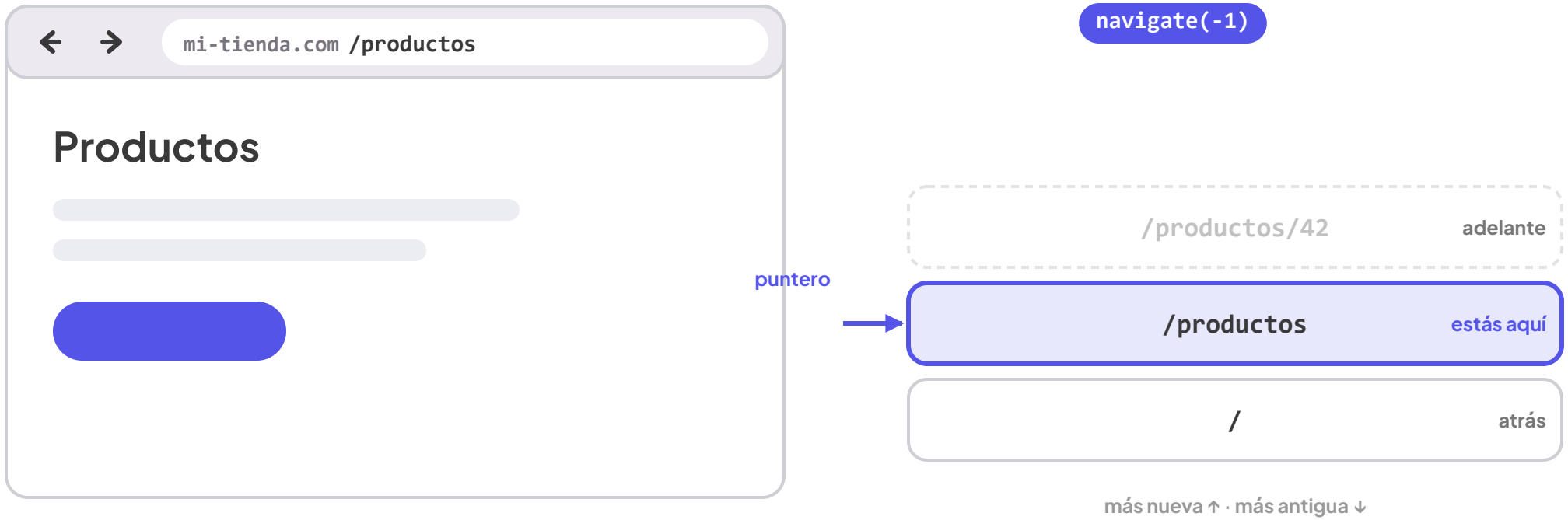
Navegar a una página nueva es un **push**: se agrega una entrada encima y el **puntero** pasa a ella.

Cuantas más páginas visitas, más alta la pila



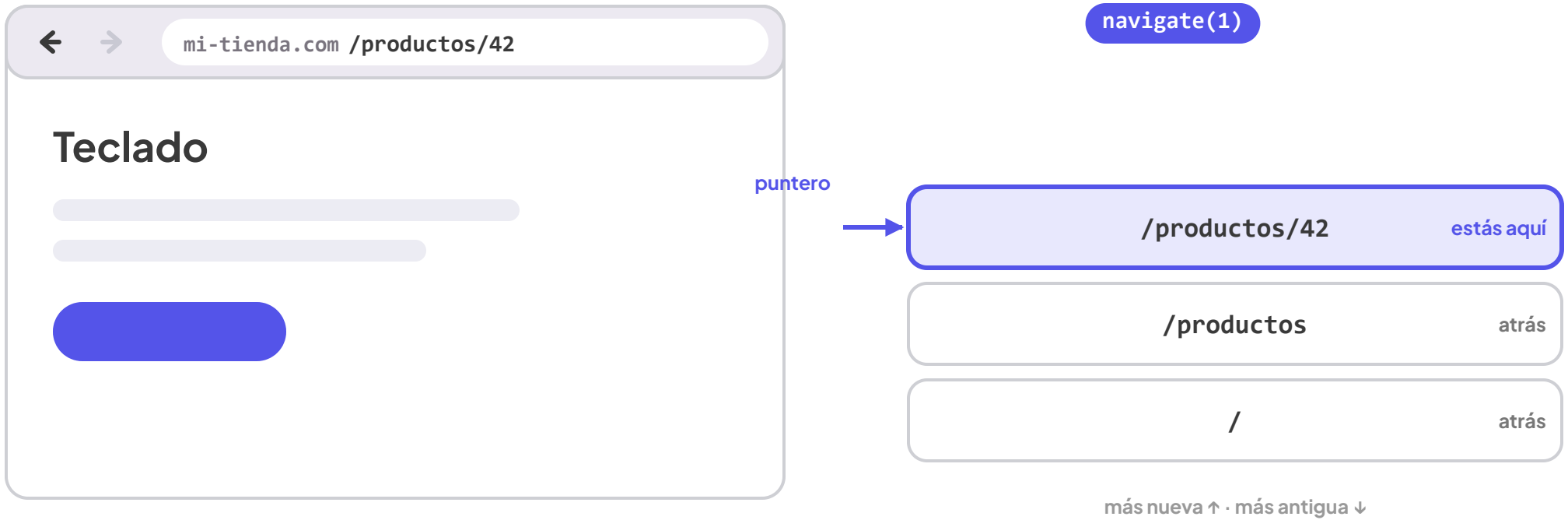
Las entradas de abajo siguen en el historial, solo que **ya no se ven**.

Atrás mueve el puntero



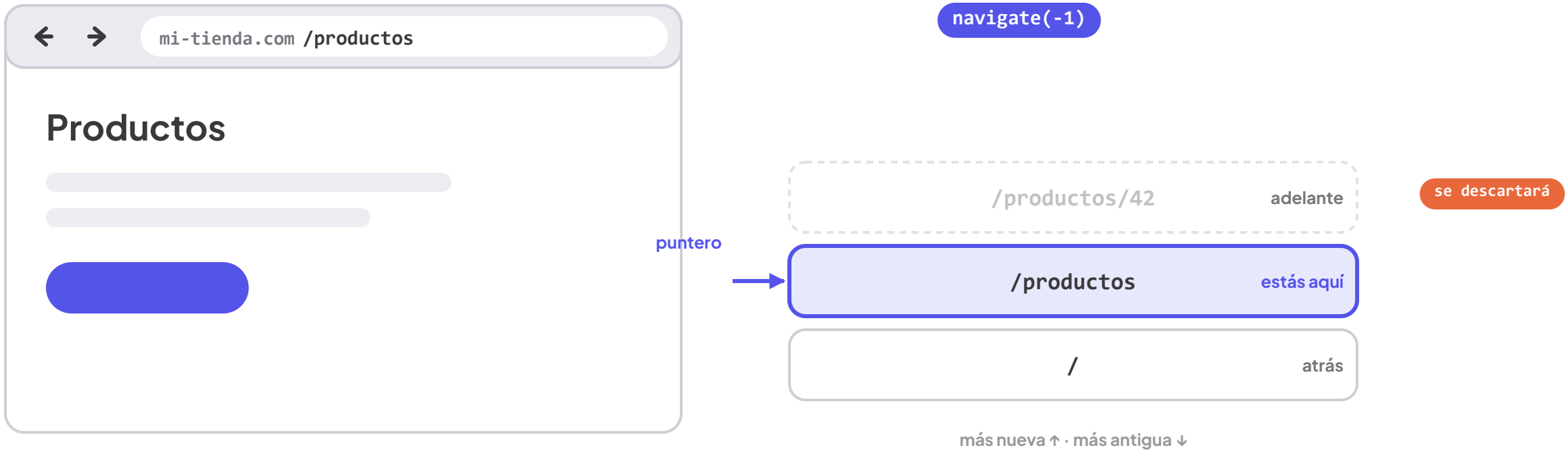
Atrás **no saca** la entrada de la pila: mueve el puntero una posición hacia abajo. La entrada de adelante queda atenuada.

Adelante lo devuelve



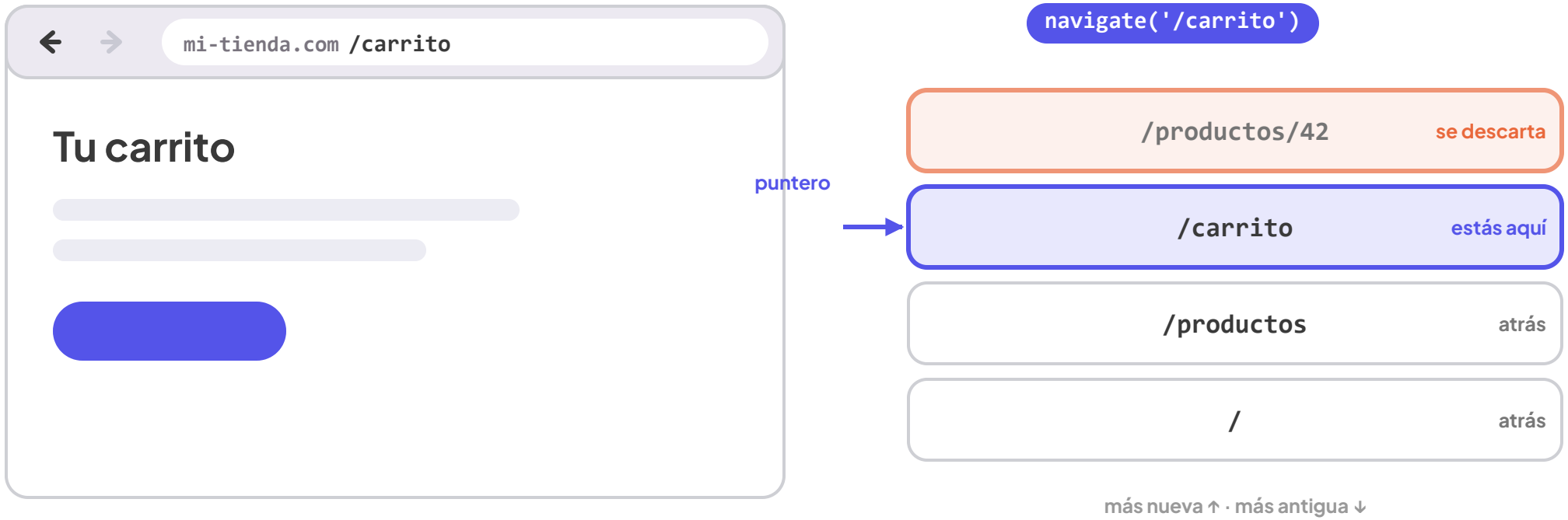
Adelante mueve el puntero hacia arriba. Solo es posible **mientras haya entradas por delante**.

Atrás, y luego navegas a otro sitio



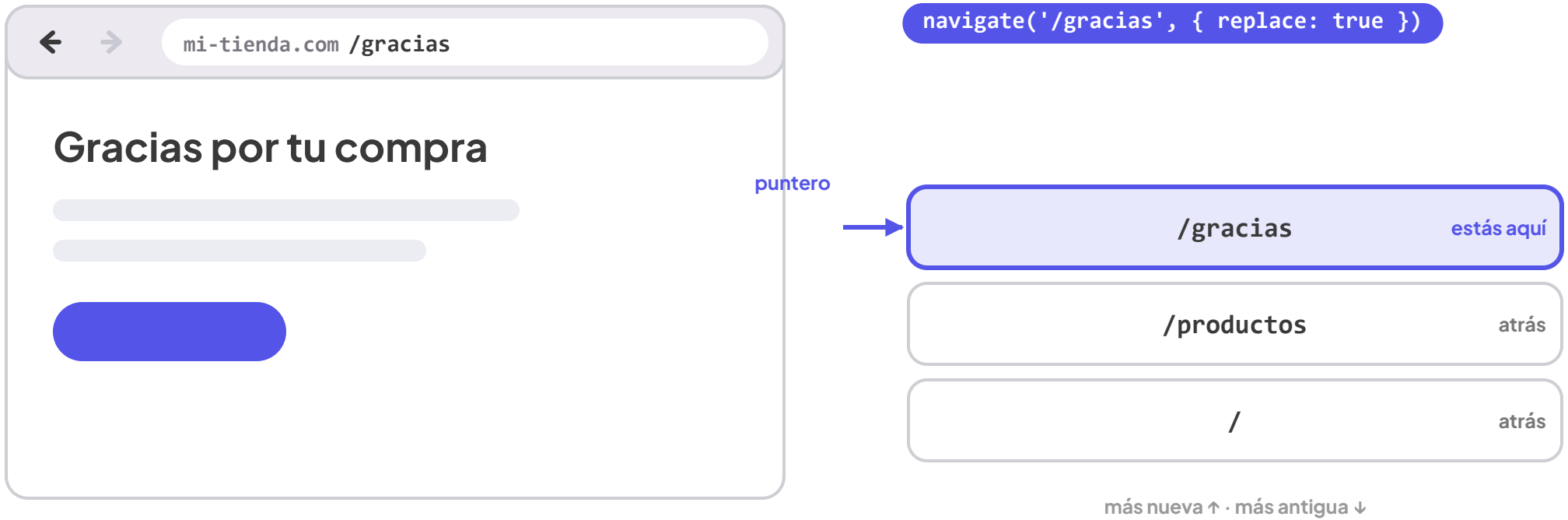
Estás en «/productos» y el Teclado sigue adelante. ¿Qué pasa si ahora abres el carrito en vez de volver al Teclado?

Un push desde el medio corta lo de adelante



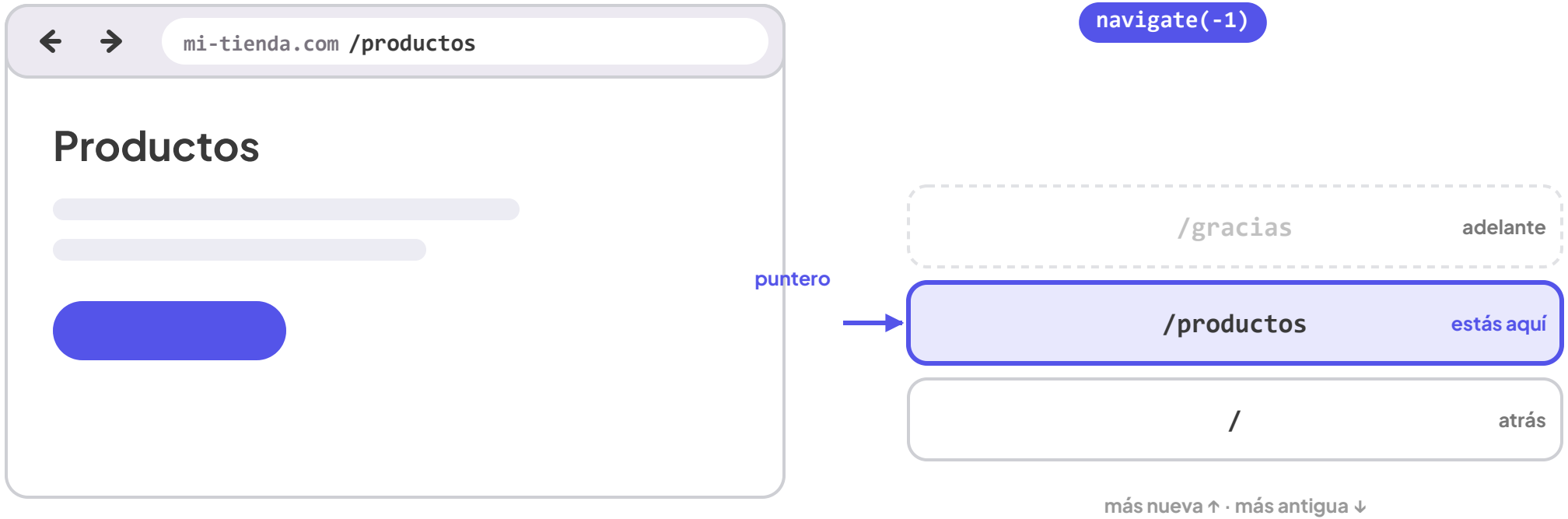
Navegar a una página nueva **descarta todo lo que había por delante** del puntero. Solo se conserva lo que está atrás.

replace reemplaza la entrada actual



Tras pagar no quieres que Atrás vuelva al carrito. Con `replace` la entrada actual se **sustituye**: el historial no crece.

Atrás salta el carrito



Atrás lleva a «/productos»: el carrito ya no está en el historial.

Qué hace cada llamada con el historial

navigate("/carrito")
Push: apila una entrada nueva. Si había entradas adelante, se descartan.

navigate(-1) / navigate(1)
Atrás / Adelante: mueve el puntero una entrada.

navigate("/gracias", { replace: true })
Replace: reemplaza la entrada actual; el historial no crece.

Cuándo usar Link y cuándo navigate

<LINK>

Cuando el usuario hace clic en algo que **es un enlace**. Es accesible y se puede abrir en otra pestaña.

NAVIGATE

Cuando la navegación es **consecuencia** de algo: terminó un envío, expiró la sesión, un botón que hace más cosas.

REPLACE: TRUE

Cuando **no quieres que Atrás vuelva** a esa página: tras pagar, iniciar sesión o una redirección.

Universidad ICESI

7. Datos en la URL

Path variables y query params

Una URL puede llevar datos



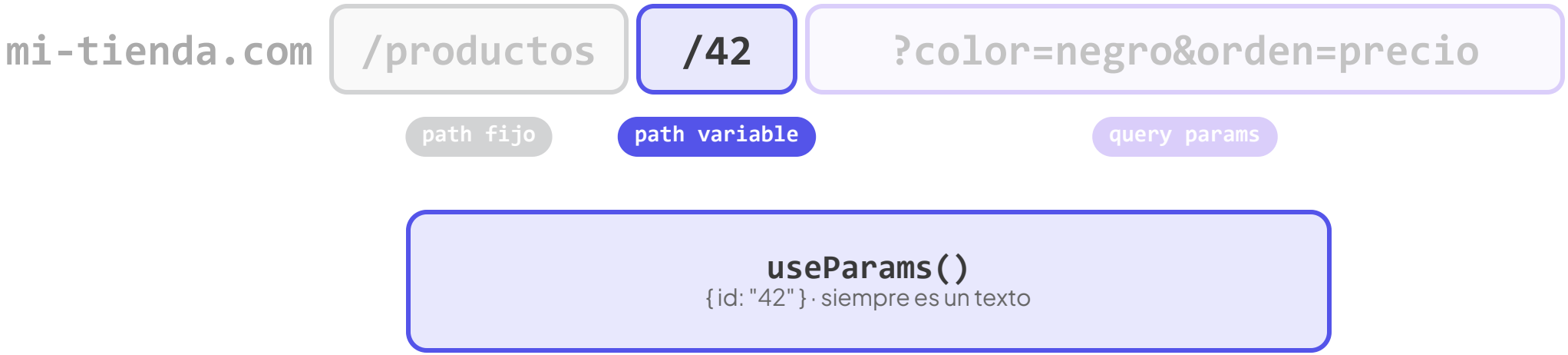
Tres partes con trabajos distintos: la ruta fija, un dato que identifica **cuál** recurso y las opciones para **cómo** verlo.

La parte fija elige la ruta



Los segmentos sin `:` deben coincidir tal cual. No cambian de una visita a otra.

Path variable: cuál recurso



El `:id` de la ruta reserva un segmento. `useParams()` lo entrega como **texto**: `"42"`, no `42`.

Query params: cómo verlo



Después del `?` van pares `clave=valor` separados por `&`. Son **opcionales**: la ruta coincide con o sin ellos.

Path variable frente a query param

PATH VARIABLE · /PRODUCTOS/42

Cuál recurso (el producto 42).

- Se define en la ruta: `productos/:id`.
- Es obligatorio: sin él la ruta no coincide.
- Se lee con `useParams()`.

QUERY PARAMS · ?CATEGORIA=TECLADOS

Cómo verlo: filtro, orden, página.

- No se definen en la ruta: son libres.
- Opcionales: la ruta coincide igual.
- Se leen con `useSearchParams()`.

Navegar con un path variable

Lista.jsx

```jsx
import { Link } from "react-router";

export default function Lista() {
  return (
    <ul>
      {productos.map((p) => (
        <li key={p.id}>
          <Link to={`/productos/${p.id}`}>
            {p.nombre}
          </Link>
        </li>
      ))}
    </ul>
  );
}
```

Detalle.jsx

```jsx
import { useParams } from "react-router";

export default function Detalle() {
  const { id } = useParams();

  return <h1>Producto {id}</h1>;
}
```

> El `id` llega como **texto**. Si lo necesitas como número, usa `Number(id)`.

Filtrar con query params

Lista.jsx

```jsx
import { useSearchParams } from "react-router";

export default function Lista() {
  const [params, setParams] = useSearchParams();
  const categoria = params.get("categoria");

  const visibles = categoria
    ? productos.filter((p) => p.categoria === categoria)
    : productos;

  return (
    <>
      <button onClick={() => setParams({ categoria: "teclados" })}>
        Teclados
      </button>
      <button onClick={() => setParams({})}>Todos</button>
    </>
  );
}
```

Se usa como useState
Devuelve el valor actual y una función para cambiarlo.

get("categoria")
Devuelve el texto o `null` si no existe.

setParams(...)
Cambia la URL y el componente se vuelve a ejecutar.

Universidad ICESI



El estado vive en la URL

Cambiar los query params **navega** a la nueva URL: `/productos?categoria=teclados`.

El filtro se puede **compartir** en un enlace y **sobrevive a una recarga**, algo que un `useState` no logra.

Universidad ICESI

8. Cargar datos

loader y manejo de errores

Un loader pide los datos antes de dibujar

src/router.jsx

```jsx
{
  path: ":id",
  loader: async ({ params }) => {
    const res = await fetch(`https://api.example.com/users/${params.id}`);
    if (!res.ok) throw new Response("No existe", { status: 404 });
    return res.json();
  },
  Component: Detalle,
  ErrorBoundary: ErrorPagina,
}
```

loader({ params })
Corre **antes** de dibujar la ruta y recibe los `params`.

Si algo falla
Lanza un error o una `Response` con su código de estado.

useLoaderData entrega los datos al componente

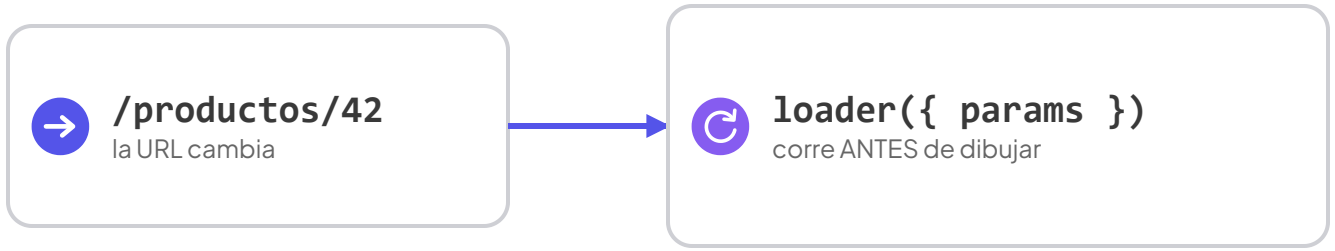
Detalle.jsx

```jsx
import { useLoaderData } from "react-router";

export default function Detalle() {
  const usuario = useLoaderData();
  return <h1>{usuario.name}</h1>;
}
```

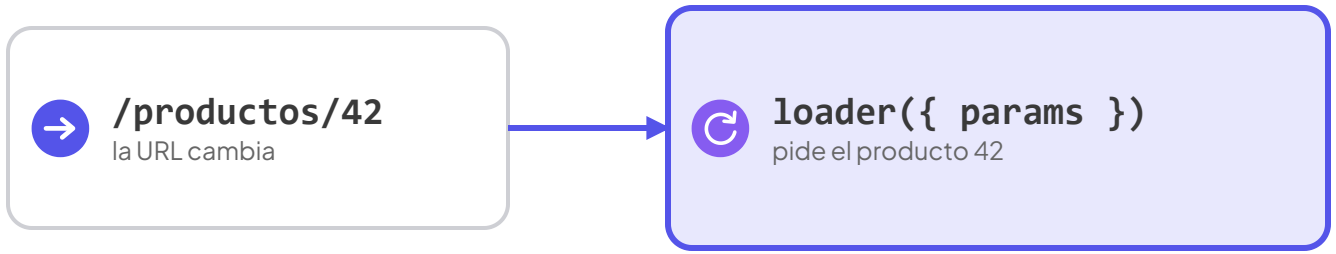
Sin useEffect
Ni estado de carga: cuando el componente se dibuja, los datos **ya están**.

La URL cambia y corre el loader



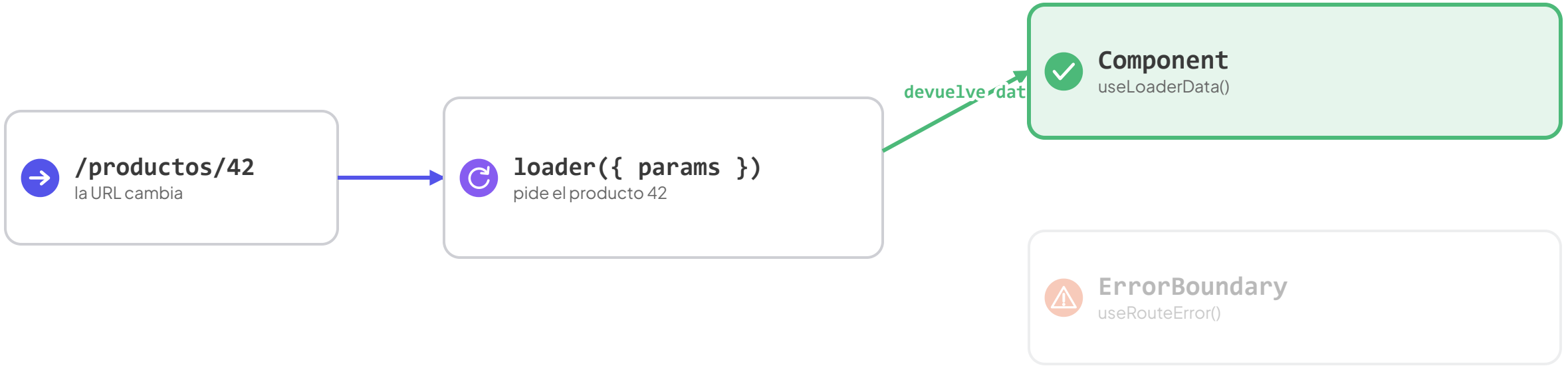
Al navegar a `/productos/42`, el router busca la ruta y ejecuta su `loader` **antes** de dibujar nada.

El loader pide los datos



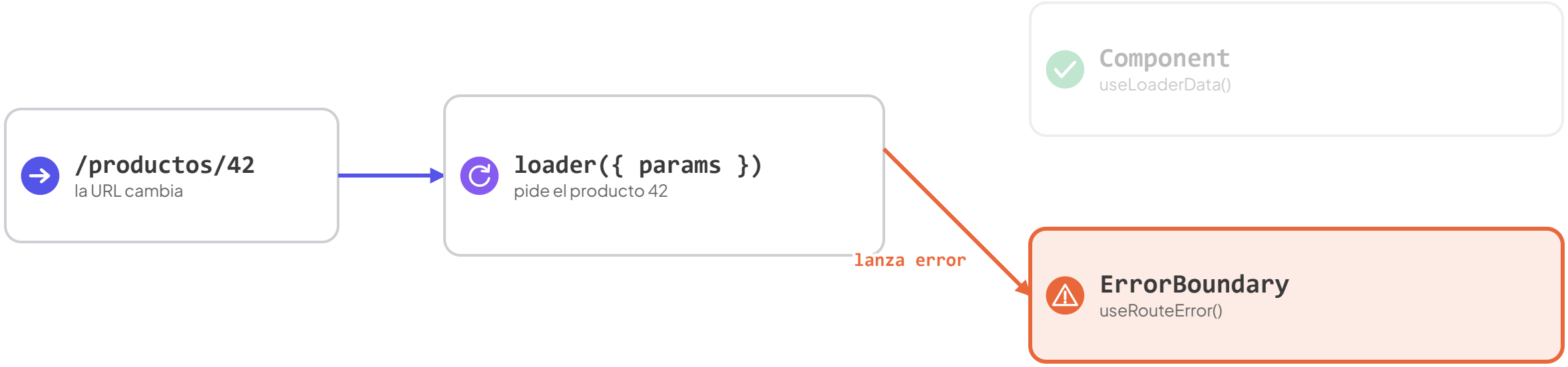
El `loader` recibe `params.id = "42"` y pide el producto. La navegación **espera** a que termine.

Si todo sale bien, se dibuja el componente



El `loader` devuelve los datos y el router dibuja el `Component`, que los lee con `useLoaderData()`.

Si el loader lanza un error, se dibuja el ErrorBoundary



Un error (o una `Response` 404) hace que el router dibuje el `ErrorBoundary` de la ruta **en lugar** del componente.

useRouteError entrega lo que se lanzó

ErrorPagina.jsx

```jsx
import { useRouteError, isRouteErrorResponse } from "react-router";

export default function ErrorPagina() {
  const error = useRouteError();

  if (isRouteErrorResponse(error)) {
    return <h1>{error.status}: {error.data}</h1>;
  }
  return <h1>Algo salió mal</h1>;
}
```

useRouteError()
Devuelve lo que lanzó el `loader`.

isRouteErrorResponse
Distingue una `Response` con código de estado de un error cualquiera.

Universidad ICESI



Cuándo loader y cuándo efecto

Un `loader` sirve para datos que **la página necesita para existir** (el producto del `:id`): se piden antes de dibujar.

`useEffect` sigue siendo la herramienta para sincronizar algo con el componente **ya en pantalla**.